07 – NoSQL

NoSQL, an alternative to SQL databases

Jérôme Landré – **jerome.landre@ju.se**

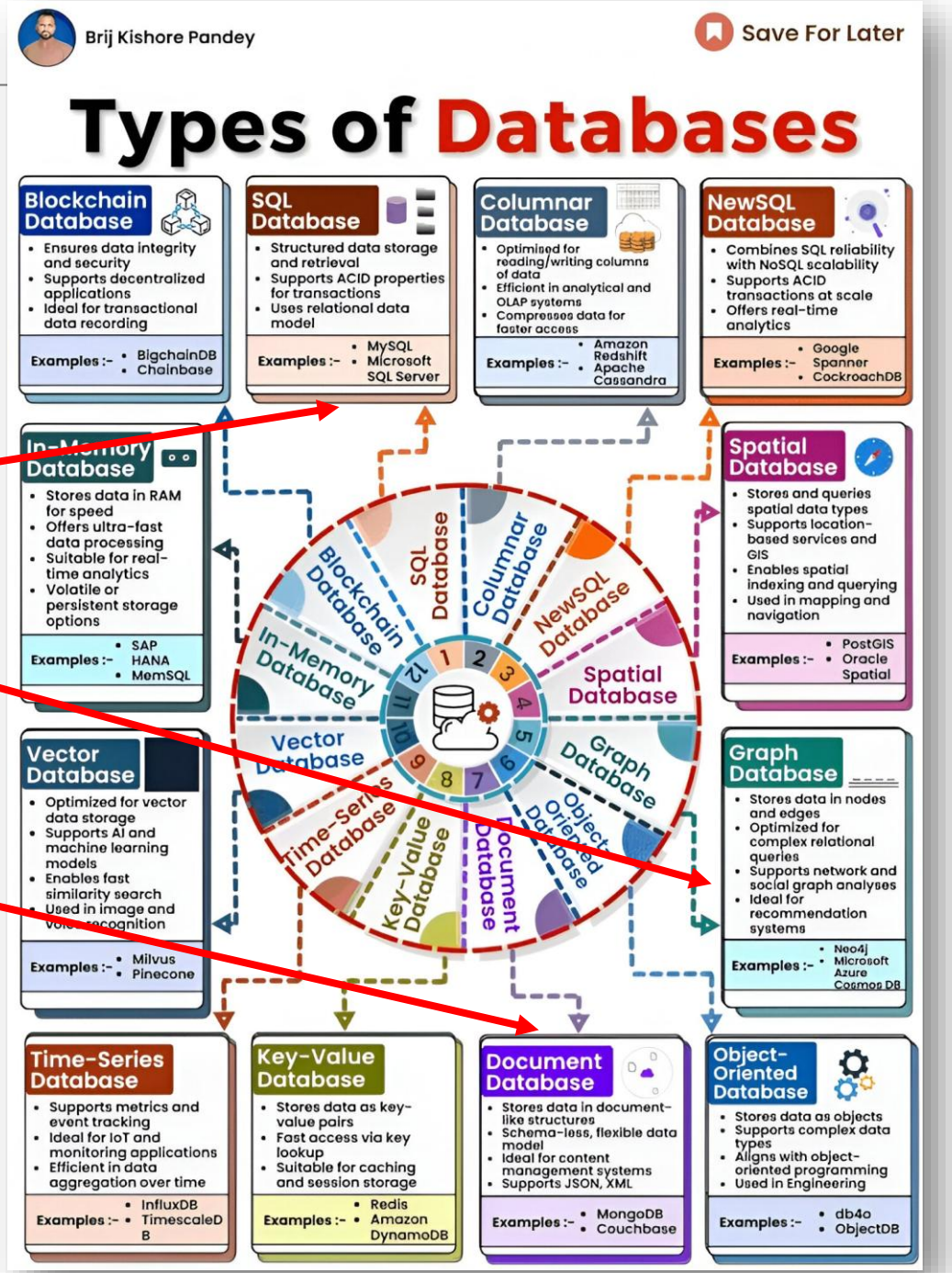
(Several slides have been created by Jasmin Jakupovic)

OUTLINE

1) Types of Databases
2) Databases rankings
3) NoSQL databases
4) Document databases
5) Graph Databases
6) Neo4J
7) Cypher

1) TYPES OF DATABASES

- There are many different types of databases, they can be sorted by type of data / type of application
- SQL Database, Graph Database, Document Database, Key-Value Database…



Source: https://www.reddit.com/r/Database/comments/1hjyvkg/types_of_databases/?rdt=47365

2) DATABASES RANKING

- Relational DB are the most represented and the best ranked

- MongoDB, Databricks, Redis, Apache Cassandra are the four NoSQL DB in positions 5, 7, 8 and 10 respectively

431 systems in ranking, April 2026

Rank Apr 2026	Rank Mar 2026	Rank Apr 2025	DBMS	Database Model	Score Apr 2026	Score Mar 2026	Score Apr 2025
1.	1.	1.	Oracle	Relational, Multi-model	1157.93	-24.53	-73.12
2.	2.	2.	MySQL	Relational, Multi-model	857.69	-0.65	-129.41
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	702.08	-9.40	-82.93
4.	4.	4.	PostgreSQL	Relational, Multi-model	681.35	+1.27	+14.10
5.	5.	5.	MongoDB	Document, Multi-model	385.02	+1.44	-15.03
6.	6.	6.	Snowflake	Relational	217.38	+6.13	+49.28
7.	7.	↑ 12.	Databricks	Multi-model	150.47	+4.67	+51.19
8.	8.	↓ 7.	Redis	Key-value, Multi-model	146.53	+1.35	-7.58
9.	9.	9.	IBM Db2	Relational, Multi-model	113.25	+1.87	-12.79
10.	↑ 11.	↑ 11.	Apache Cassandra	Wide column, Multi-model	102.62	+0.74	-5.59
11.	↓ 10.	↓ 8.	Elasticsearch	Multi-model	99.51	-4.07	-28.57
12.	12.	↓ 10.	SQLite	Relational	95.90	-0.08	-18.19
13.	13.	↑ 14.	MariaDB	Relational, Multi-model	86.41	-0.59	-7.93
14.	14.	↑ 16.	Microsoft Azure SQL Database	Relational, Multi-model	75.96	+2.03	+0.59
15.	15.	↑ 18.	Apache Hive	Relational	74.13	+1.18	+1.48
16.	16.	↑ 17.	Splunk	Search engine	73.08	+0.87	-0.31
17.	17.	↓ 13.	Microsoft Access	Relational	64.92	-2.65	-30.45
18.	18.	↓ 15.	Amazon DynamoDB	Multi-model	62.67	-2.75	-13.82
19.	19.	19.	Google BigQuery	Relational	52.36	-3.46	-8.37
20.	20.	20.	Neo4j	Graph	46.88	-0.44	-0.71
21.	↑ 22.	↑ 23.	SAP HANA	Relational, Multi-model	33.23	+1.14	-0.17
22.	↓ 21.	↑ 24.	Apache Solr	Search engine, Multi-model	33.17	+0.38	+0.77
23.	23.	↓ 22.	Teradata	Relational, Multi-model	31.11	+0.03	-2.97
24.	24.	↓ 21.	FileMaker	Relational	28.81	-0.50	-7.67
25.	25.	25.	SAP Adaptive Server	Relational, Multi-model	25.31	-0.22	-3.48
26.	↑ 28.	↑ 31.	ClickHouse	Relational, Multi-model	22.87	+0.86	+4.14
27.	↓ 26.	↓ 26.	Microsoft Azure Cosmos DB	Multi-model	22.43	-0.31	-0.12
28.	↓ 27.	↑ 29.	Apache Spark (SQL)	Relational	21.97	-0.10	+1.57
29.	29.	↑ 30.	PostGIS	Spatial, Multi-model	21.51	-0.36	+1.90
30.	↑ 31.	↑ 32.	OpenSearch	Multi-model	20.73	+0.46	+3.54

Source: https://db-engines.com/en/ranking

3) NOSQL DATABASES

NoSQL stands for:

- Not Just SQL
- No SQL Database

This type of database emerged from unmet needs. The internet was getting too big (too fast) for relational databases.

The name itself implies that there is no structure in the database... but is there?

But Why?

When we have a problem, we often try to use existing solutions. If they do not meet our needs, we try to develop something that does.

When specialized applications started demanding faster processing, NoSQL happened.

3) NOSQL DATABASES

Since it was developed to solve a specific problem, it had to bring some benefits to the table.

Larger web applications, such as online stores and social networks, had problems with the implementation and usage of relational databases.

Only 4 reasons were enough for NoSQL to exist and persist.

Four reasons why:

- **Scalability**
- **Cost**
- **Flexibility**
- **Availability**

Some features might be more important than others; depending on your application needs.

3) NOSQL DATABASES

Scalability:
Being able to meet the needs of various workloads is always a benefit. If you don't really need all the power all the time why pay for it?

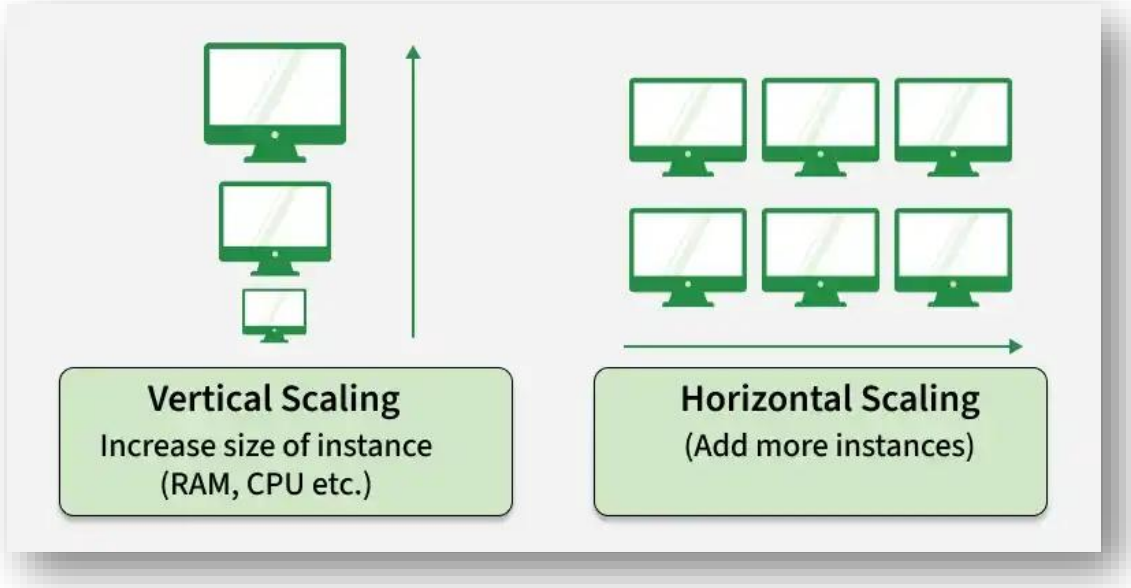
For businesses/applications to grow they need to scale. There are two ways of scaling:

- Scaling Out
- Scaling Up

Scaling Out/Horizontal Scaling:
Adding resources per demand is called scaling out. Relational databases have trouble scaling out. Scaling out is adjusting to the current demand and then, going back.

Scaling Up/Vertical Scaling:
Is done by adding more RAM and CPU's. It also requires downtime since you have to make copies before you scale.

3) NOSQL DATABASES



Scaling Out/Horizontal Scaling:
Adding resources per demand is called scaling out. Relational databases have trouble scaling out. Scaling out is adjusting to the current demand and then, going back.

Scaling Up/Vertical Scaling:
Is done by adding more RAM and CPU's. It also requires downtime since you have to make copies before you scale.

Source: https://www.geeksforgeeks.org/system-design/system-design-horizontal-and-vertical-scaling/

3) NOSQL DATABASES

Cost:

When you use SQL solutions on such big scales the cost of software licensing increases.

There is also the cost of scaling, which is much easier done when scaling out vs scaling up.

NoSQL solutions are most often open source. Some companies offer open source software but charge for support, scaling or simply by usage.

How drastic is it?

On small applications the cost difference might be minimum but as the application grows costs grow as well.

Most of the cost is actually related to scalability but also to flexibility and availability.

3) NOSQL DATABASES

Flexibility:
Every DB administrator that worked with databases long enough experienced the headache of constraints, relations, tables.

We are, in essence, as flexible as our model supports us. You have probably learned this in Lab 3 when you start defining new tables and deleting old ones or change the datatype of an attribute.

Know your database...
With SQL you need to know your Database structure pretty much to work with it.

NoSQL – NoProblems :)
Some NoSQL solutions require no fixed table structures and are very flexible. You can add and remove as much as you want…

3) NOSQL DATABASES

Availability:
Time is money.

The more time your database is down the more money you waste. With updates and maintenance, you often have to put your database down.

If your up-time is bad users tend to be hesitant to visit you again.

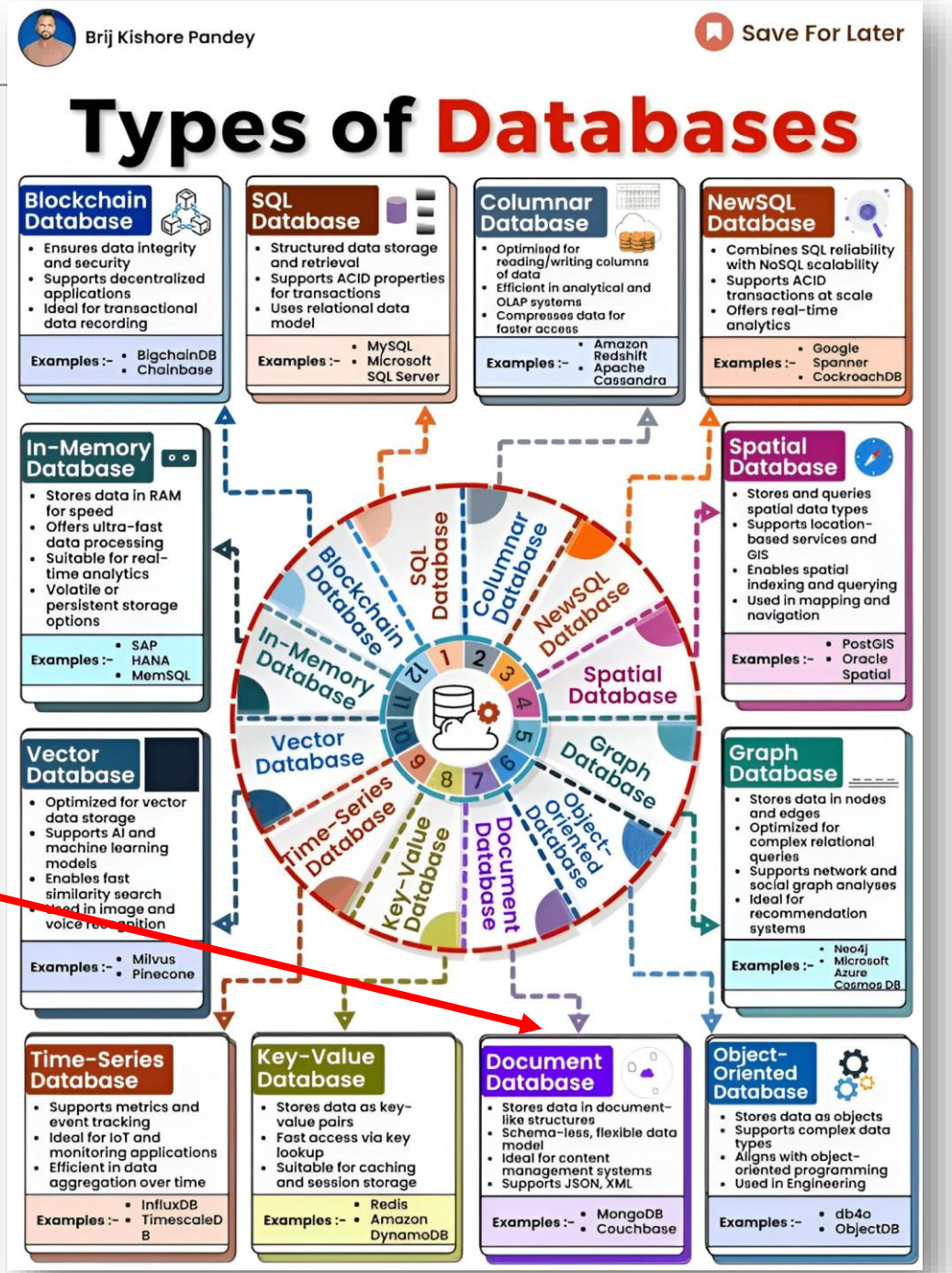
NoSQL is available!

NoSQL databases can run on almost anything, they:

- have low-cost servers,
- work in clusters (one down, others take over),
- might suffer on performance but will be up!

5) GRAPH DATABASES

- There are many different types of databases, they can be sorted by type of data / type of application
- SQL Database, Graph Database, Document Database, Key-Value Database…



Source: https://www.reddit.com/r/Database/comments/1hjyvkg/types_of_databases/?rdt=47365

4) DOCUMENT DATABASES

- A well-known (and well-ranked) document database is **MongoDB.** From MongoDB official site:

"**Document databases** offer a variety of advantages, including:
- An **intuitive data model** that is **fast** and **easy** for developers to work with
- A **flexible schema** that allows for the **data model to evolve** as application needs change
- The **ability** to horizontally **scale out**

Because of these advantages, document databases are general-purpose databases that can be used in a variety of use cases and industries.
Document databases are considered to be non-relational (or NoSQL) databases. Instead of storing data in fixed rows and columns, document databases use flexible documents. Document databases are the most popular alternative to tabular, relational databases."

4) DOCUMENT DATABASES

- From MongoDB official site:

What are documents?

A document is a record in a document database. A document typically stores information about one object and any of its related metadata.

Documents store data in field-value pairs. The values can be a variety of types and structures, including strings, numbers, dates, arrays, or objects. Documents can be stored in formats like JSON, BSON, and XML.

4) DOCUMENT DATABASES

What is JSON?

JSON stands for JavaScript Object Notation but it is used in many more languages than Javascript because of its easy and powerful syntax.

To write JSON data, you **ONLY** need **7 characters**!

[… **]**: array
{ … **}**: object
"key"**:**"value": a key/value pair separated by :
" … **"**: a character string
,: separator between objects and key/value pairs

All JSON is there:

[ ] { } : " ,

4) DOCUMENT DATABASES

What is BSON?

BSON stands for Binary JavaScript Object Notation (used by MongoDB): JSON is encoded as binary values:

JSON:

```
{
  "name": "Jerome Landre",
  "age": 40,
  "isStudent": false,
  "address": {
    "city": "Stockholm",
    "zip": "12345"
  },
  "hobbies": ["reading", "hiking", "coding"]
}
```

BSON:

```
5A 00 00 00 02 6E 61 6D 65 00 0D 00 00 00 4A 65 72 6F 6D 65 20 4C 61 6E 64 72 65 00 10 61
67 65 00 28 00 00 00 08 69 73 53 74 75 64 65 6E 74 00 00 03 61 64 64 72 65 73 73 00 2E 00
00 00 02 63 69 74 79 00 07 00 00 00 53 74 6F 63 6B 68 6F 6C 6D 00 02 7A 69 70 00 06 00 00
00 31 32 33 34 35 00 04 68 6F 62 62 69 65 73 00 1F 00 00 00 02 72 65 61 64 69 6E 67 00 08
00 00 00 02 68 69 6B 69 6E 67 00 08 00 00 00 02 63 6F 64 69 6E 67 00 00 00
```

4) DOCUMENT DATABASES

*	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	

JSON:

```
{
  "name": "Jerome Landre",
  "age": 40,
  "isStudent": false,
  "address": {
    "city": "Stockholm",
    "zip": "12345"
  },
  "hobbies": ["reading", "hiking", "coding"]
}
```

BSON:

```
5A 00 00 00 02 6E 61 6D 65 00 0D 00 00 00 4A 65 72 6F 6D 65 20 4C 61 6E 64 72 65 00 10 61
67 65 00 28 00 00 00 08 69 73 53 74 75 64 65 6E 74 00 00 03 61 64 64 72 65 73 73 00 2E 00
00 00 02 63 69 74 79 00 07 00 00 00 53 74 6F 63 6B 68 6F 6C 6D 00 02 7A 69 70 00 06 00 00
00 31 32 33 34 35 00 04 68 6F 62 62 69 65 73 00 1F 00 00 00 02 72 65 61 64 69 6E 67 00 08
00 00 00 02 68 69 6B 69 6E 67 00 08 00 00 00 02 63 6F 64 69 6E 67 00 00 00
```

Source: http://www.maxi-pedia.com/code+ASCII

4) DOCUMENT DATABASES

Example of JSON data:

A JSON document that stores information about a user named Tom.

```json
{
  "id": 1,
  "firstName": "Tom",
  "lastName": "Sawyer",
  "age": 17,
  "gender": "male",
  "address": {
    "city": "St. Petersburg",
    "state": "Missouri",
    "country": "USA"
  },
  "hobbies": ["fishing", "adventures", "painting fences"],
  "friends": ["Huckleberry Finn", "Becky Thatcher", "Joe Harper"]
}
```

4) DOCUMENT DATABASES

Collections

A collection is a group of documents. Collections typically store documents that have similar contents.

Not all documents in a collection are required to have the same fields, because document databases have flexible schemas. Note that some document databases provide schema validation, so the schema can optionally be locked down when needed.

Continuing with the example above, the document with information about Tom could be stored in a collection named users. More documents could be added to the users collection in order to store information about other users. For example, the document below that stores information about Donna could be added to the users collection.

```
{
  "id": 1,
  "firstName": "Tom",
  "lastName": "Sawyer",
  "age": 17,
  "gender": "male",
  "address": {
    "city": "St. Petersburg",
    "state": "Missouri",
    "country": "USA"
  },
  "hobbies": ["fishing", "adventures", "painting fences"],
  "friends": ["Huckleberry Finn", "Becky Thatcher", "Joe Harper"]
}
```

```
{
  "id": 2,
  "firstName": "Rebecca",
  "lastName": "Thatcher",
  "nickName": "Becky",
  "age": 16,
  "gender": "female",
  "father": "Judge Thatcher",
  "school": "St. Petersburg Sunday School",
  "bestFriend": "Tom Sawyer",
  "personalityTraits": ["intelligent", "playful", "loyal"]
}
```

Collection “users”


```
[
    {
    "id": 1,
    "firstName": "Tom",
    "lastName": "Sawyer",
    "age": 17,
    "gender": "male",
    "address": {
        "city": "St. Petersburg",
        "state": "Missouri",
        "country": "USA"
    },
    "hobbies": ["fishing", "adventures", "painting fences"],
    "friends": ["Huckleberry Finn", "Becky Thatcher", "Joe Harper"]
    },
    {
    "id": 2,
    "firstName": "Rebecca",
    "lastName": "Thatcher",
    "nickName": "Becky",
    "age": 16,
    "gender": "female",
    "father": "Judge Thatcher",
    "school": "St. Petersburg Sunday School",
    "bestFriend": "Tom Sawyer",
    "personalityTraits": ["intelligent", "playful", "loyal"]
    }
]
```

4) DOCUMENT DATABASES

CRUD operations

Document databases typically have an API or query language that allows developers to execute the CRUD (create, read, update, and delete) operations.

- **Create**: Documents can be created in the database. Each document has a unique identifier.
- **Read**: Documents can be read from the database. The API or query language allows developers to query for documents using their unique identifiers or field values. Indexes can be added to the database in order to increase read performance.
- **Update**: Existing documents can be updated — either in whole or in part.
- **Delete**: Documents can be deleted from the database.collections

4) DOCUMENT DATABASES

What are the key features of document databases?

Document databases have the following key features:

- **Document model**: Data is stored in documents (unlike other databases that store data in structures like tables or graphs). Documents map to objects in most popular programming languages, which allows developers to rapidly develop their applications.
- **Flexible schema**: Document databases have flexible schemas, meaning that not all documents in a collection need to have the same fields. Note that some document databases support schema validation, so the schema can be optionally locked down.
- **Distributed and resilient**: Document databases are distributed, which allows for horizontal scaling (typically cheaper than vertical scaling) and data distribution. Document databases provide resiliency through replication.
- **Querying through an API or query language**: Document databases have an API or query language that allows developers to execute the CRUD operations on the database. Developers have the ability to query for documents based on unique identifiers or field values.

4) DOCUMENT DATABASES

MongoDB cheatsheet

Create Operations

Create or insert operations add new **documents** to a **collection**. If the collection does not currently exist, insert operations will create the collection.

MongoDB provides the following methods to insert documents into a collection:

- `db.collection.insertOne()`
- `db.collection.insertMany()`

In MongoDB, insert operations target a single **collection**. All write operations in MongoDB are **atomic** on the level of a single **document**.

```
db.users.insertOne(          <-- collection
   {
      name: "sue",           <-- field: value
      age: 26,               <-- field: value
      status: "pending"      <-- field: value
   }                         } document
)
```

Source: https://www.mongodb.com/docs/manual/crud/#std-label-crud

4) DOCUMENT DATABASES

MongoDB cheatsheet

Read Operations

Read operations retrieve **documents** from a **collection**; i.e. query a collection for documents. MongoDB provides the following methods to read documents from a collection:

- `db.collection.find()`

You can specify **query filters or criteria** that identify the documents to return.

```
db.users.find(                          <-- collection
   { age: { $gt: 18 } },                <-- query criteria
   { name: 1, address: 1 }              <-- projection
).limit(5)                              <-- cursor modifier
```

Source: https://www.mongodb.com/docs/manual/crud/#std-label-crud

4) DOCUMENT DATABASES

MongoDB cheatsheet

Update Operations

Update operations modify existing **documents** in a **collection**. MongoDB provides the following methods to update documents of a collection:

- `db.collection.updateOne()`
- `db.collection.updateMany()`
- `db.collection.replaceOne()`

In MongoDB, update operations target a single collection. All write operations in MongoDB are **atomic** on the level of a single document.

You can specify criteria, or filters, that identify the documents to update. These **filters** use the same syntax as read operations.

```
db.users.updateMany(                    <-- collection
   { age: { $lt: 18 } },                <-- update filter
   { $set: { status: "reject" } }       <-- update action
)
```

Source: https://www.mongodb.com/docs/manual/crud/#std-label-crud

4) DOCUMENT DATABASES

MongoDB cheatsheet

Delete Operations

Delete operations remove documents from a collection. MongoDB provides the following methods to delete documents of a collection:

- `db.collection.deleteOne()`
- `db.collection.deleteMany()`

In MongoDB, delete operations target a single **collection**. All write operations in MongoDB are **atomic** on the level of a single document.

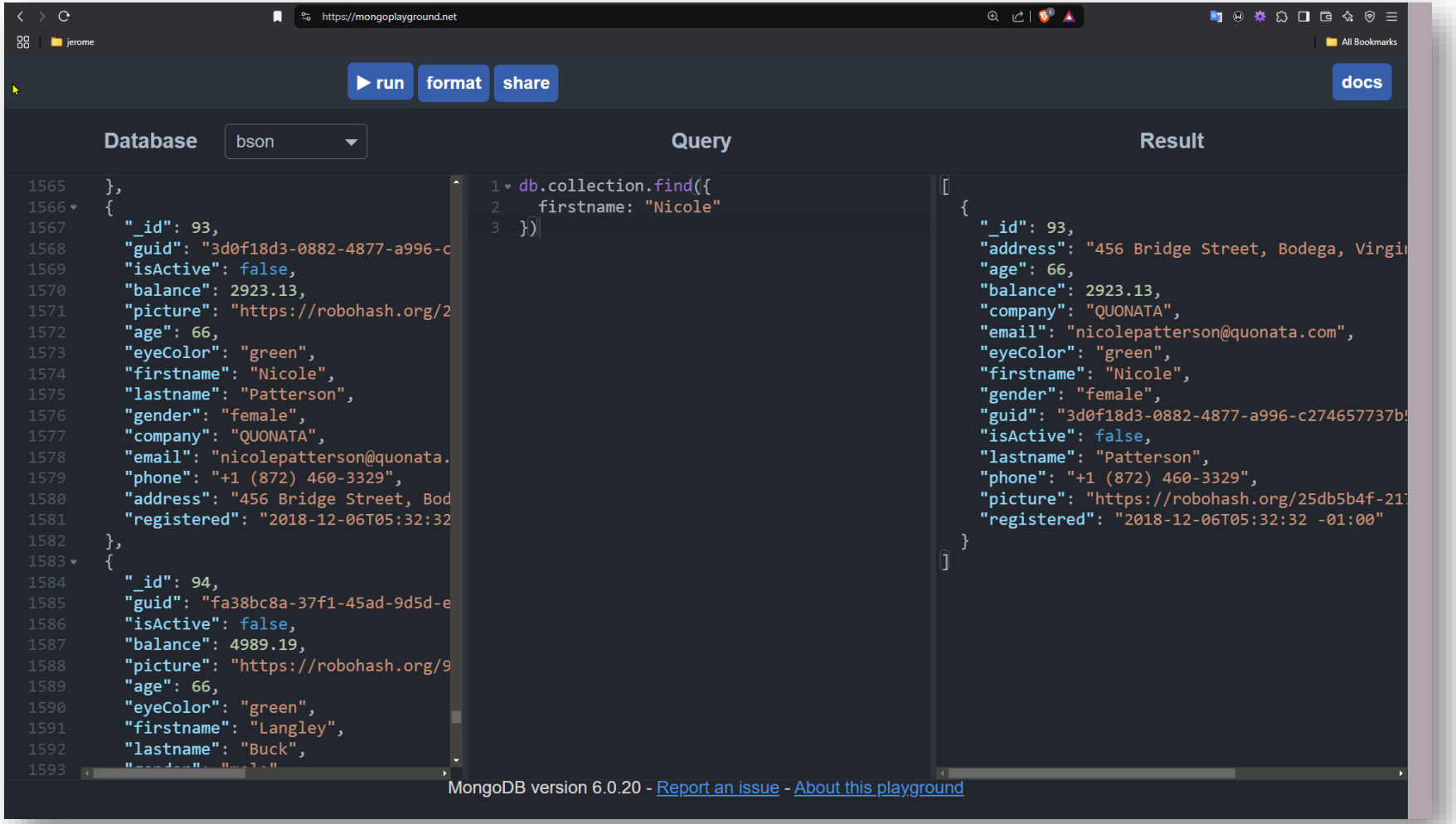
You can specify criteria, or filters, that identify the documents to remove. These **filters** use the same syntax as read operations.

```
db.users.deleteMany(          <-- collection
   { status: "reject" }       <-- delete filter
)
```

Source: https://www.mongodb.com/docs/manual/crud/#std-label-crud

4) DOCUMENT DATABASES

MongoDB playground

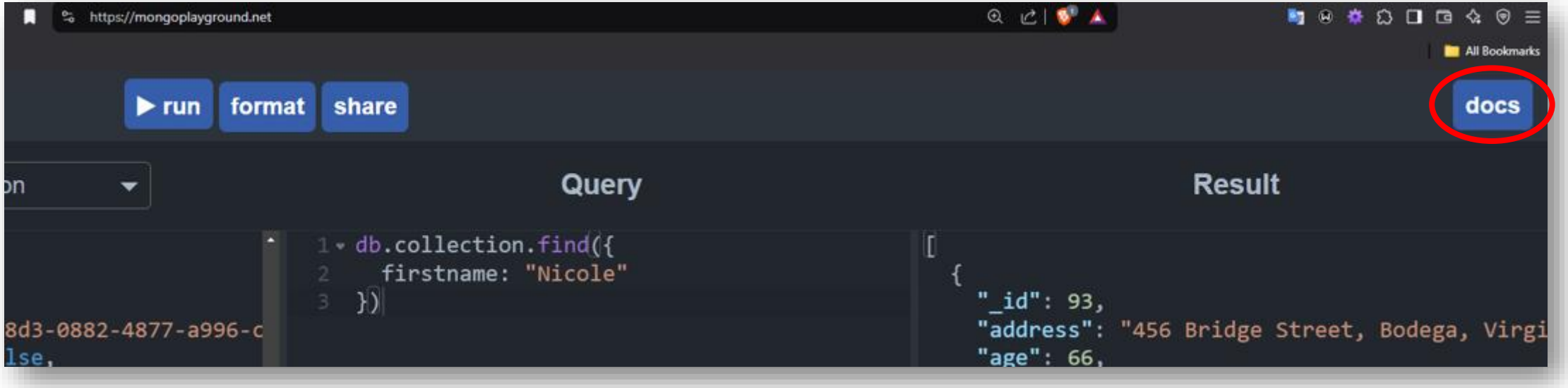


Source: https://mongoplayground.net/

4) DOCUMENT DATABASES

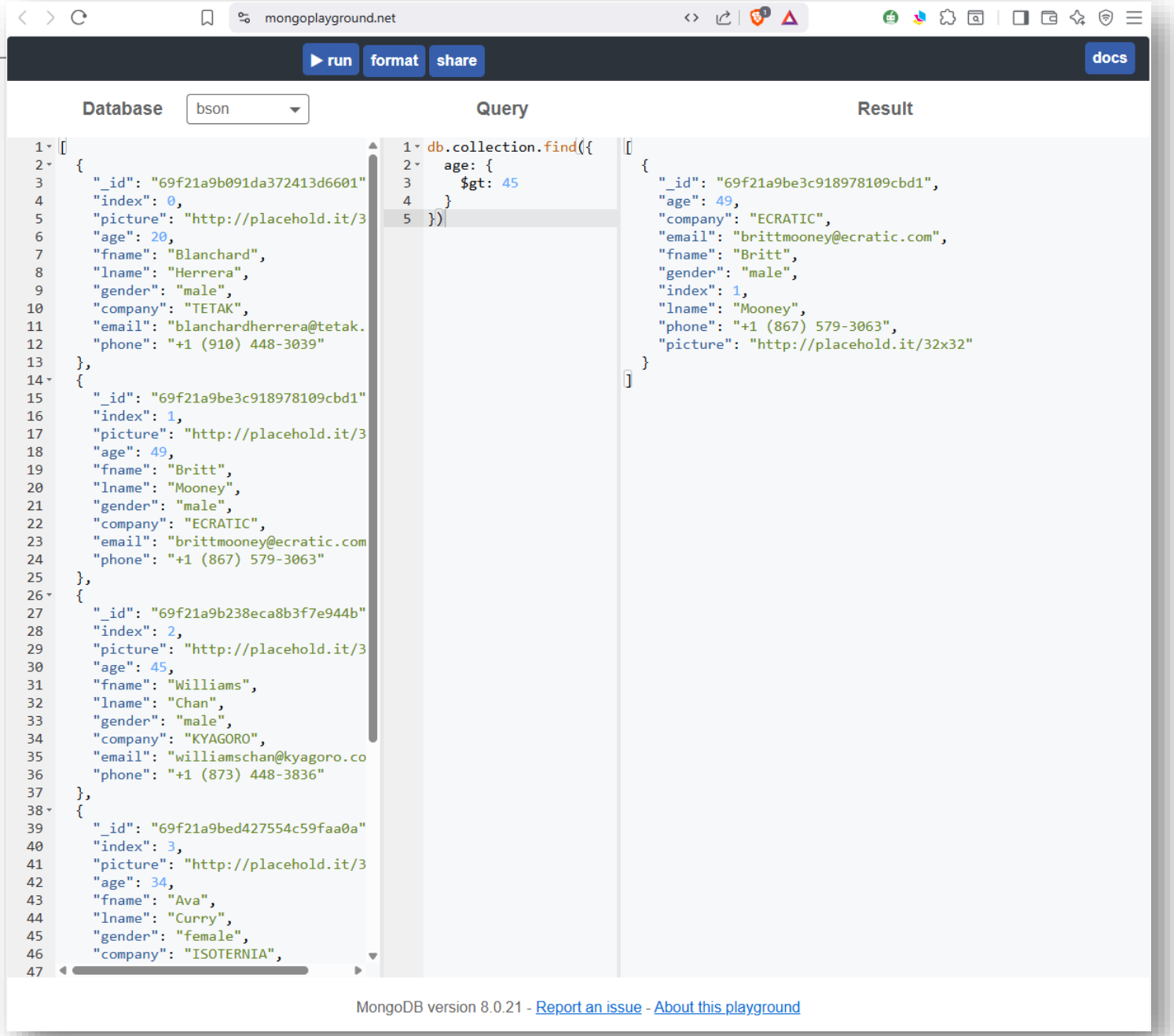
MongoDB playground

To understand, please read the



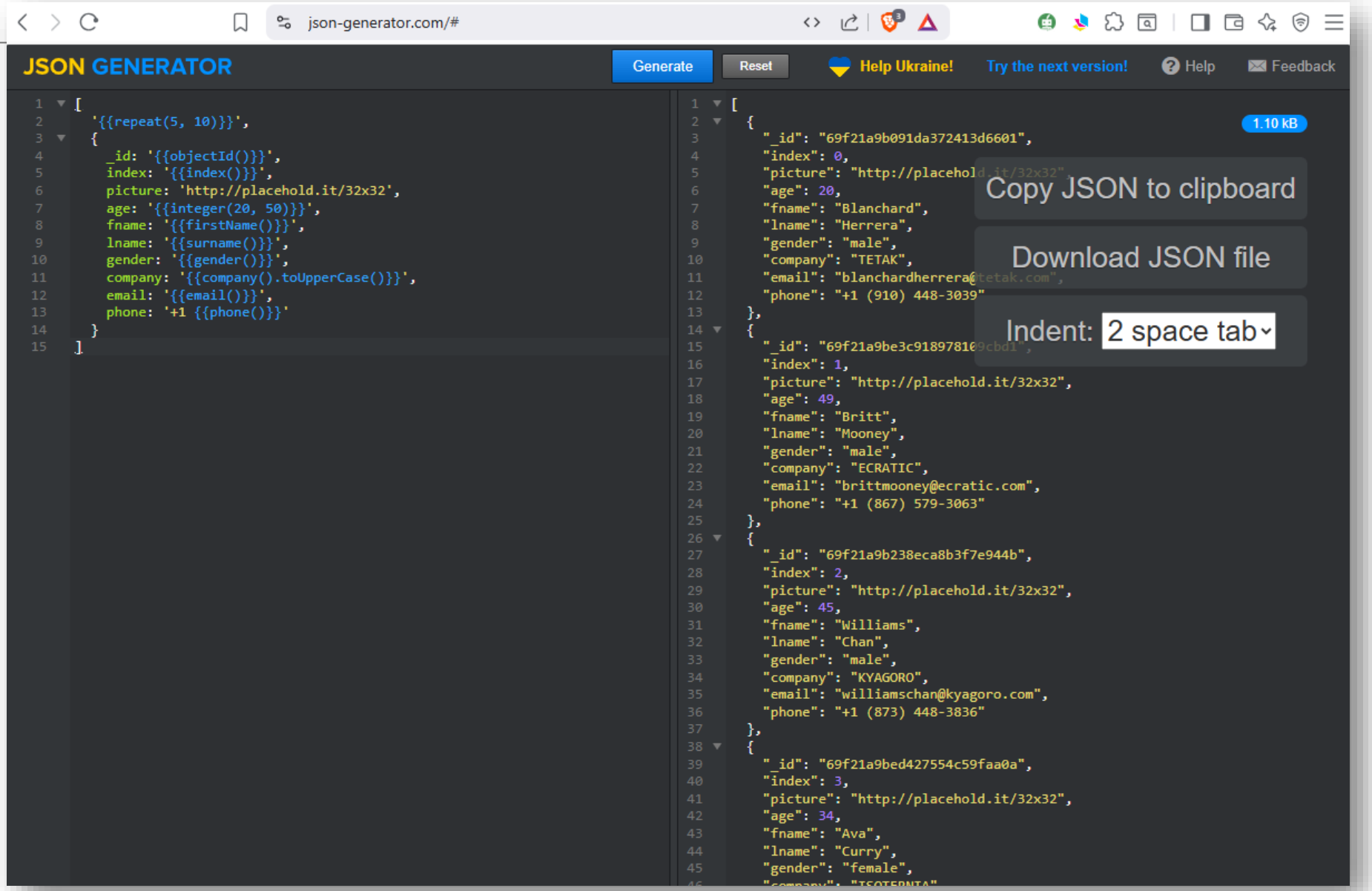
Source: https://mongoplayground.net/

MongoDB playground



Source: https://mongoplayground.net/

Generate JSON data



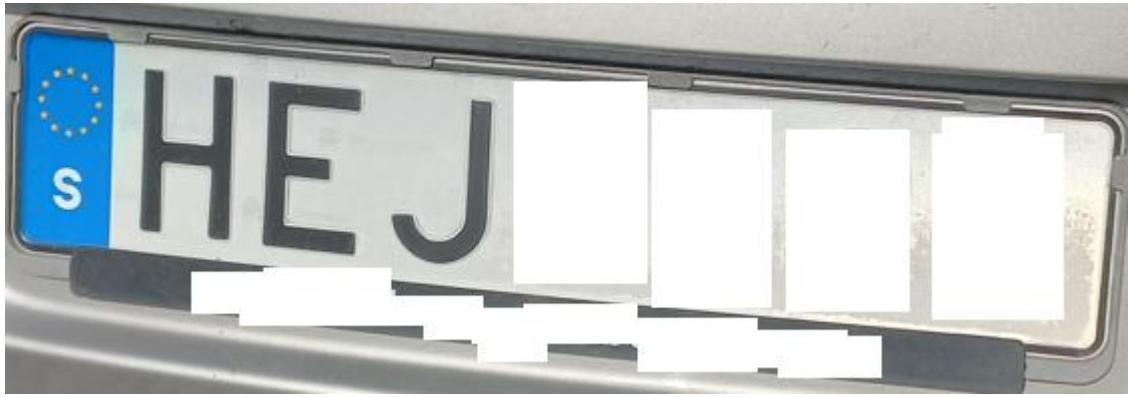
Source: https://json-generator.com

4) DOCUMENT DATABASES

Let's go for a live demonstration!

Source: https://mongoplayground.net/

In french:



Bonjour, ça va ?	/bɔ̃.ʒuʁ | sa va/
Oui, ça va bien, merci !	/wi | sa va bjɛ̃ | mɛʁ.si/
Et toi ?	/e twa/
Ça va, merci.	/sa va/

Source: Translated to IPA, not the beer, the alphabet (https://en.wikipedia.org/wiki/International_Phonetic_Alphabet) by Mistral AI (https://chat.mistral.ai/chat/).

REMEMBER: BE POSITIVE!

- Mylène Farmer, “Désenchantée” (live version):

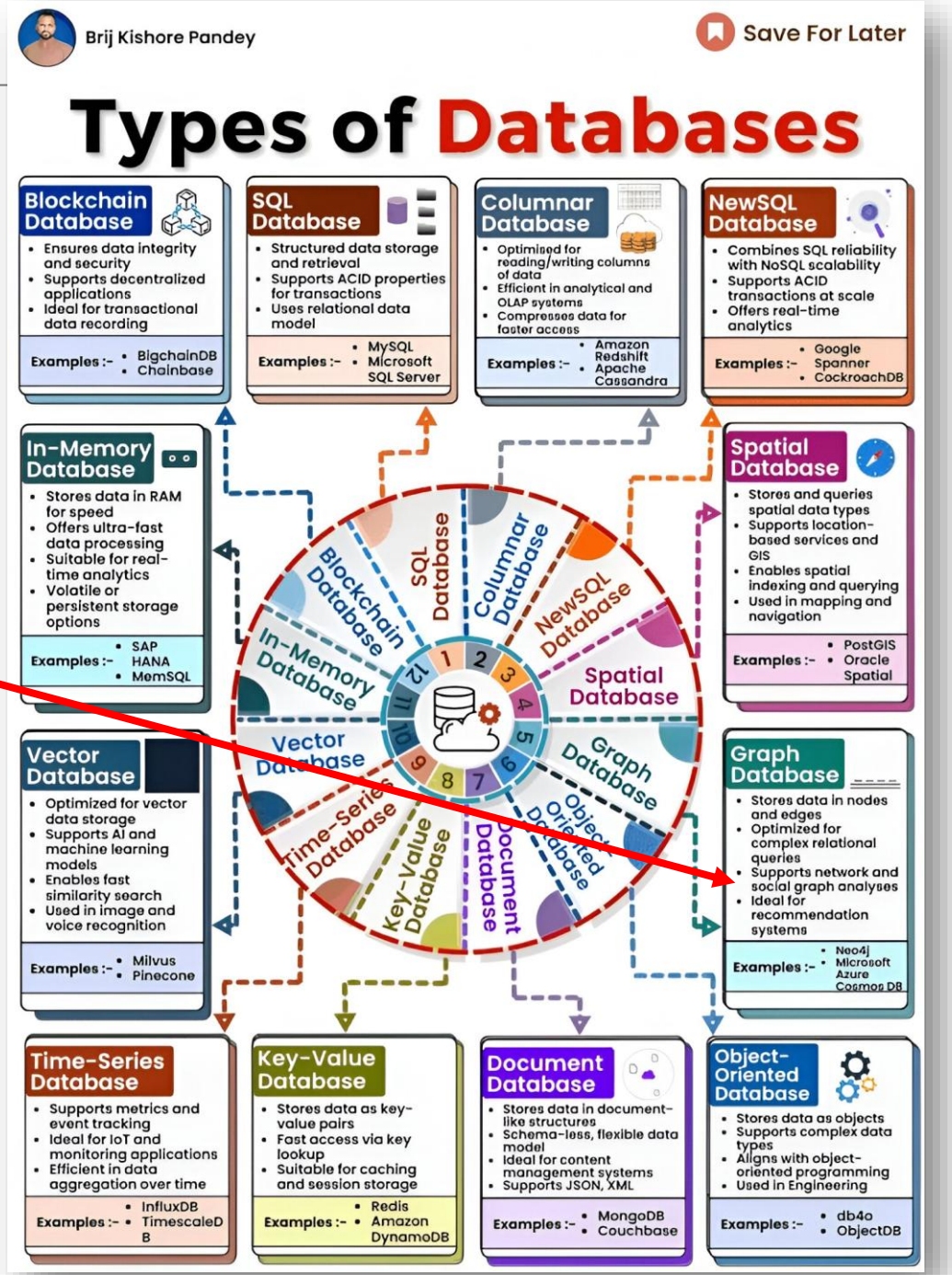
https://www.youtube.com/watch?v=r1Le-dAocI8

- Lyrics (English and Swedish translation):

https://lyricstranslate.com/en/desenchantee-disenchanted.html#songtranslation
https://lyricstranslate.com/en/desenchantee-besviken.html

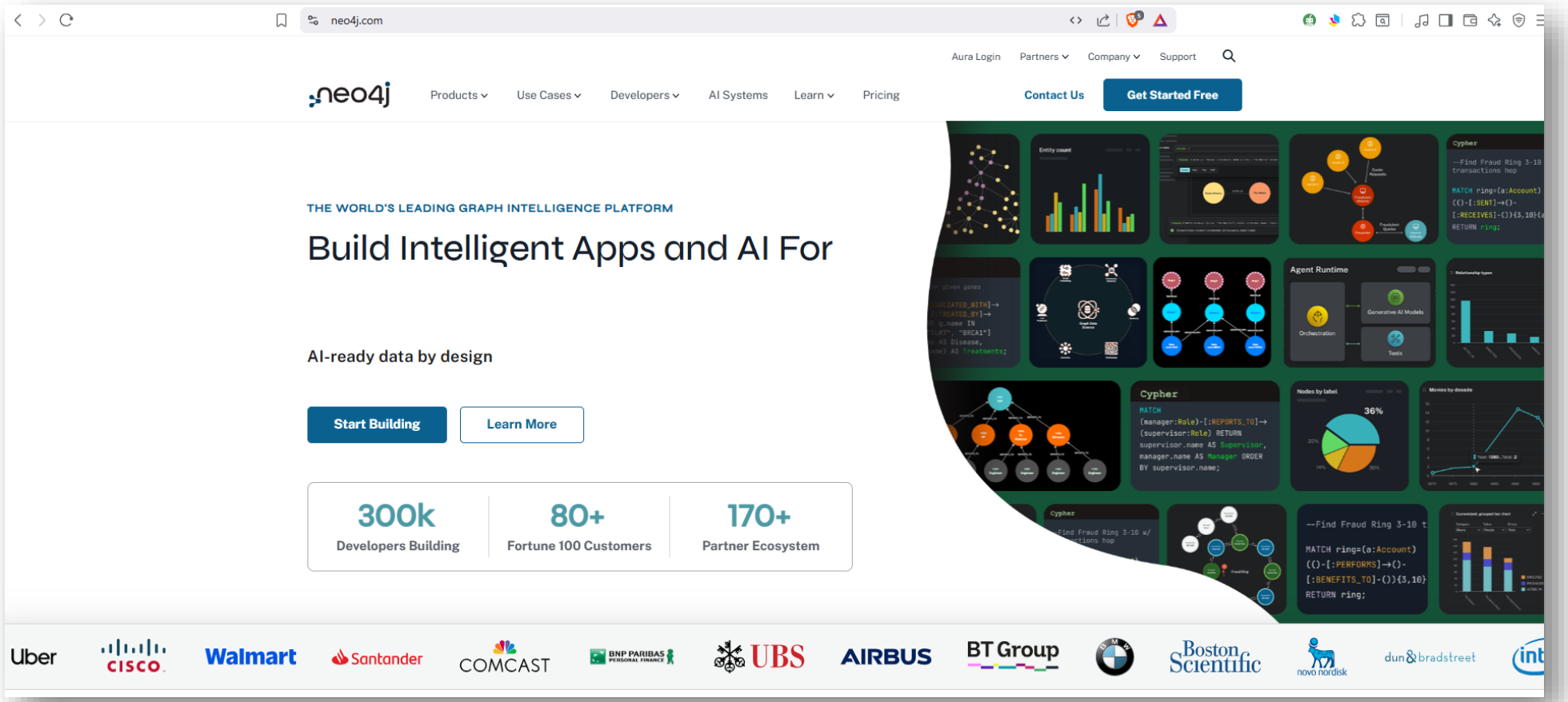
5) GRAPH DATABASES

- There are many different types of databases, they can be sorted by type of data / type of application
- SQL Database, Graph Database, Document Database, Key-Value Database



Source: https://www.reddit.com/r/Database/comments/1hjyvkg/types_of_databases/?rdt=47365

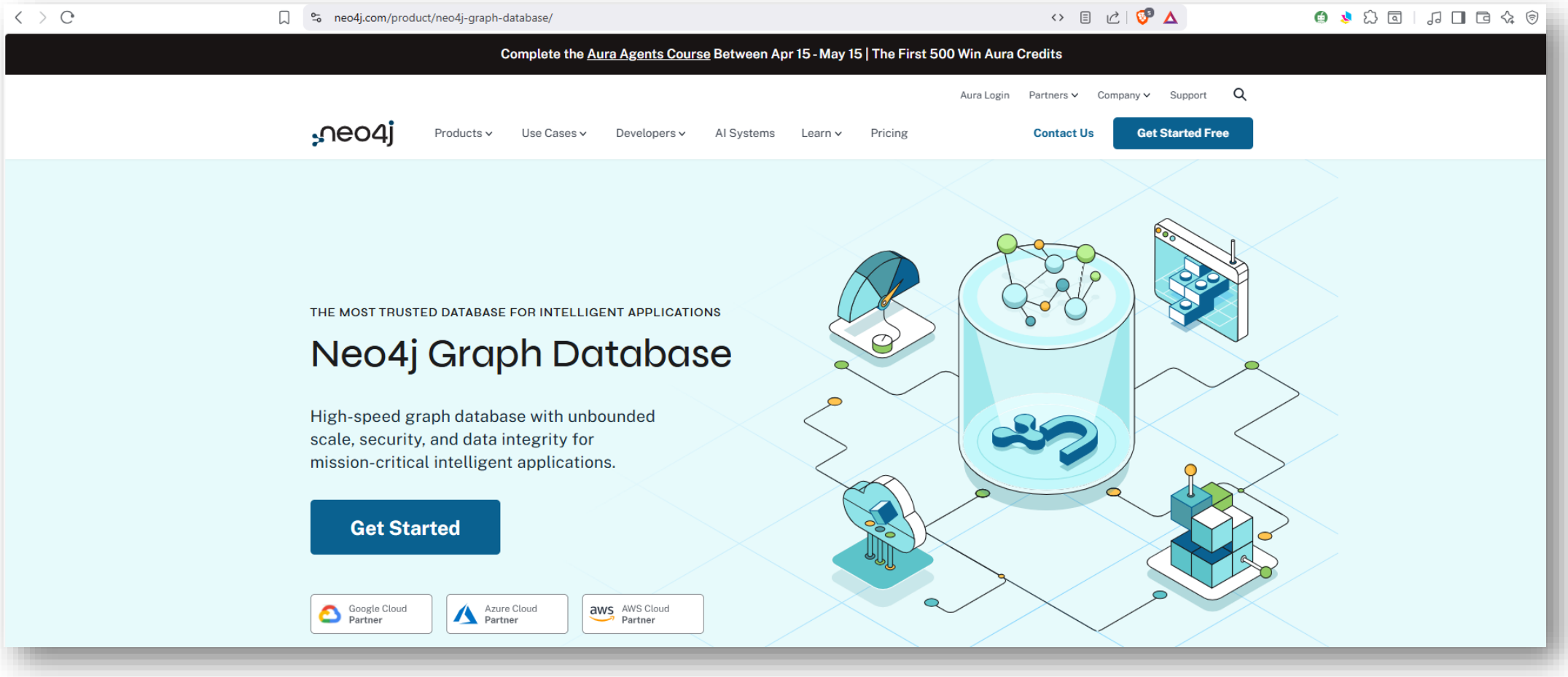
6) NEO4J



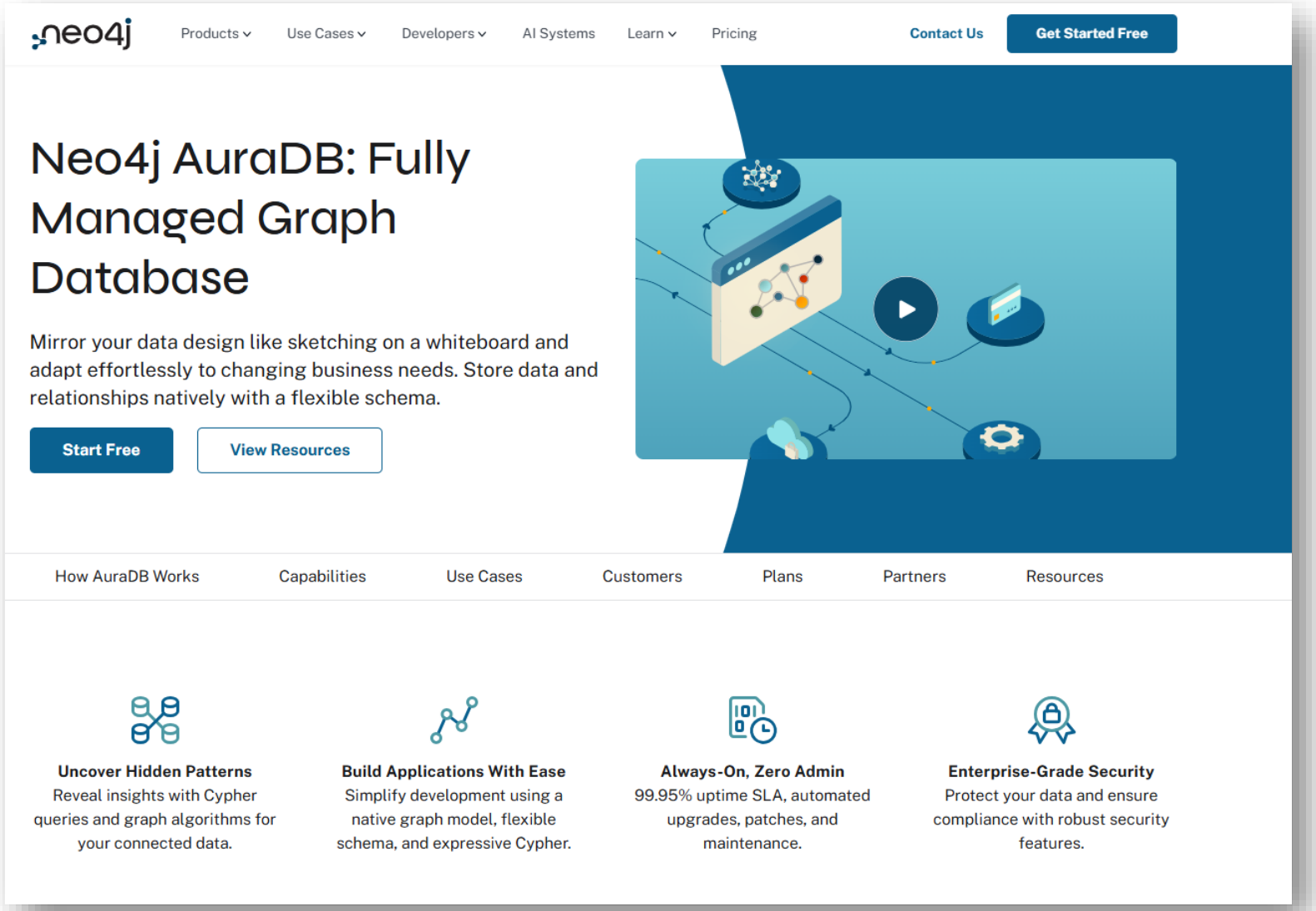
Source: https://neo4j.com/

6) NEO4J

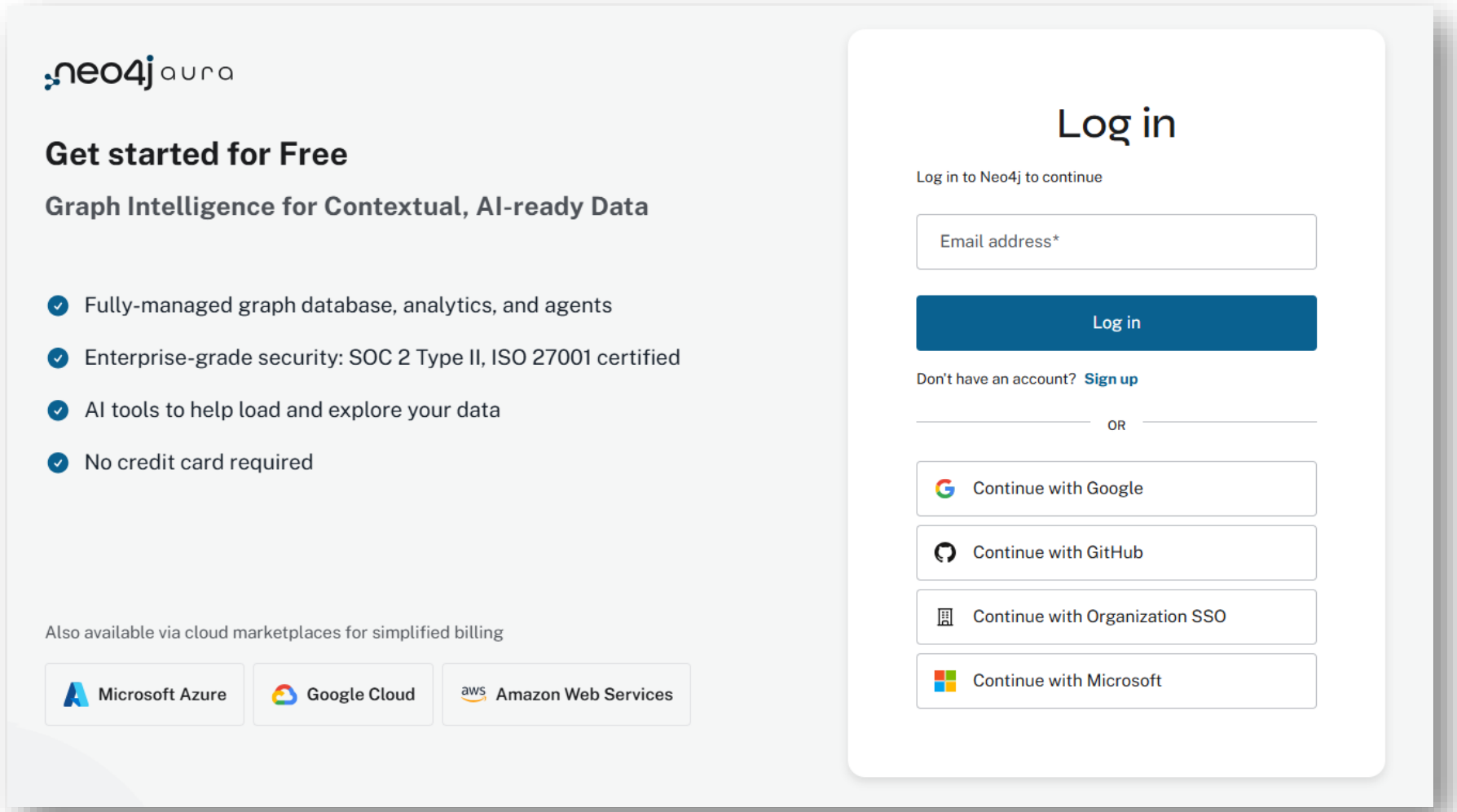
Graph Database

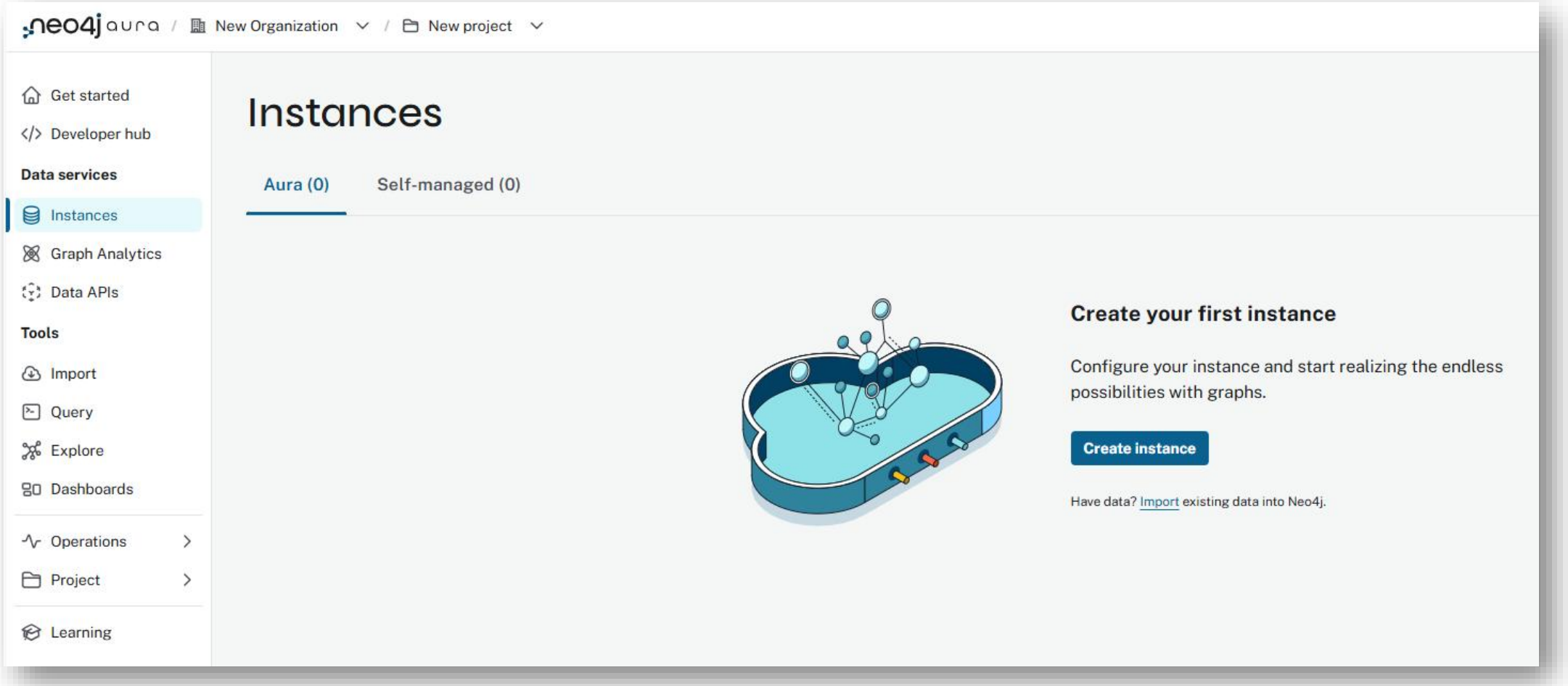


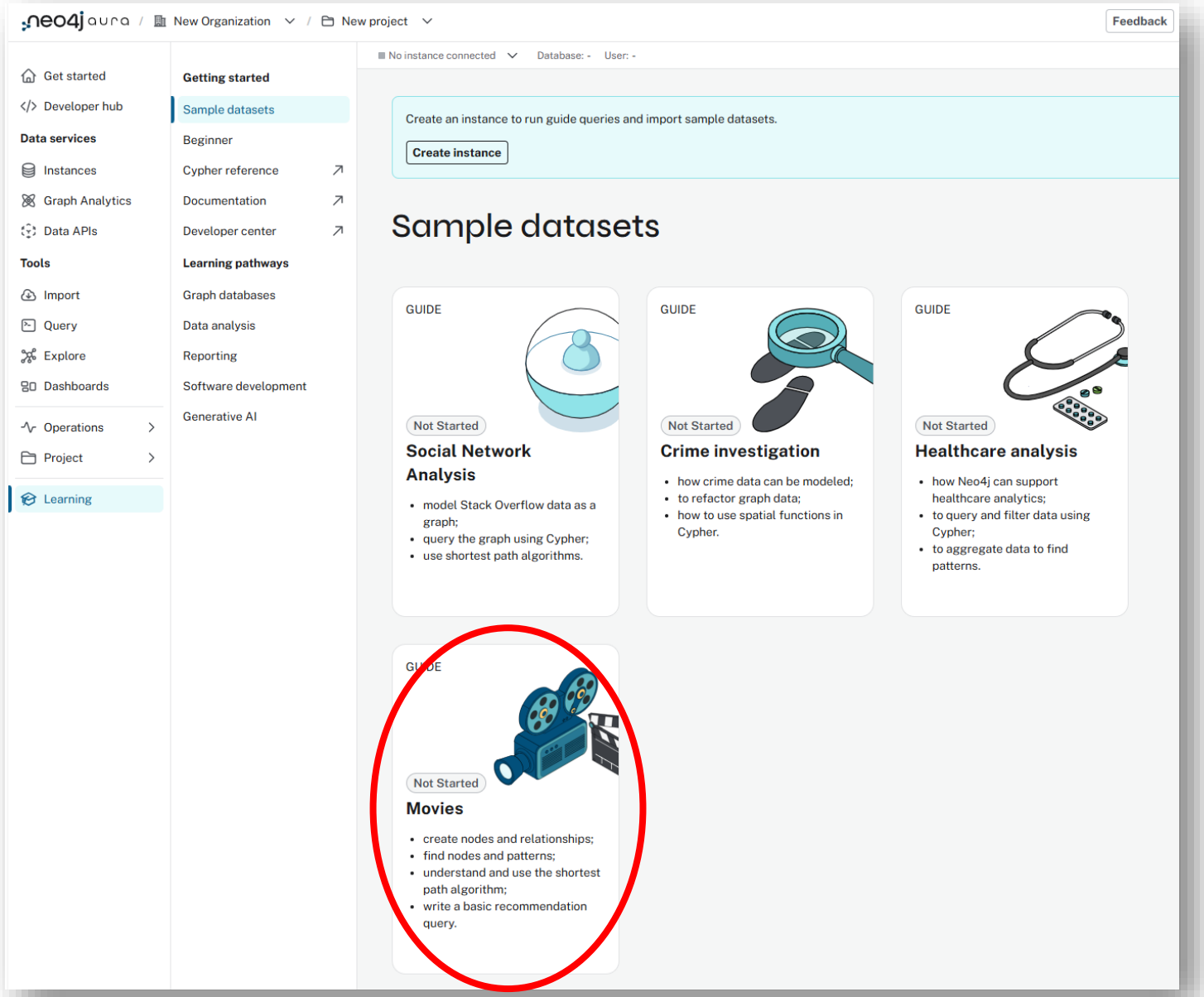
Source: https://neo4j.com/product/neo4j-graph-database

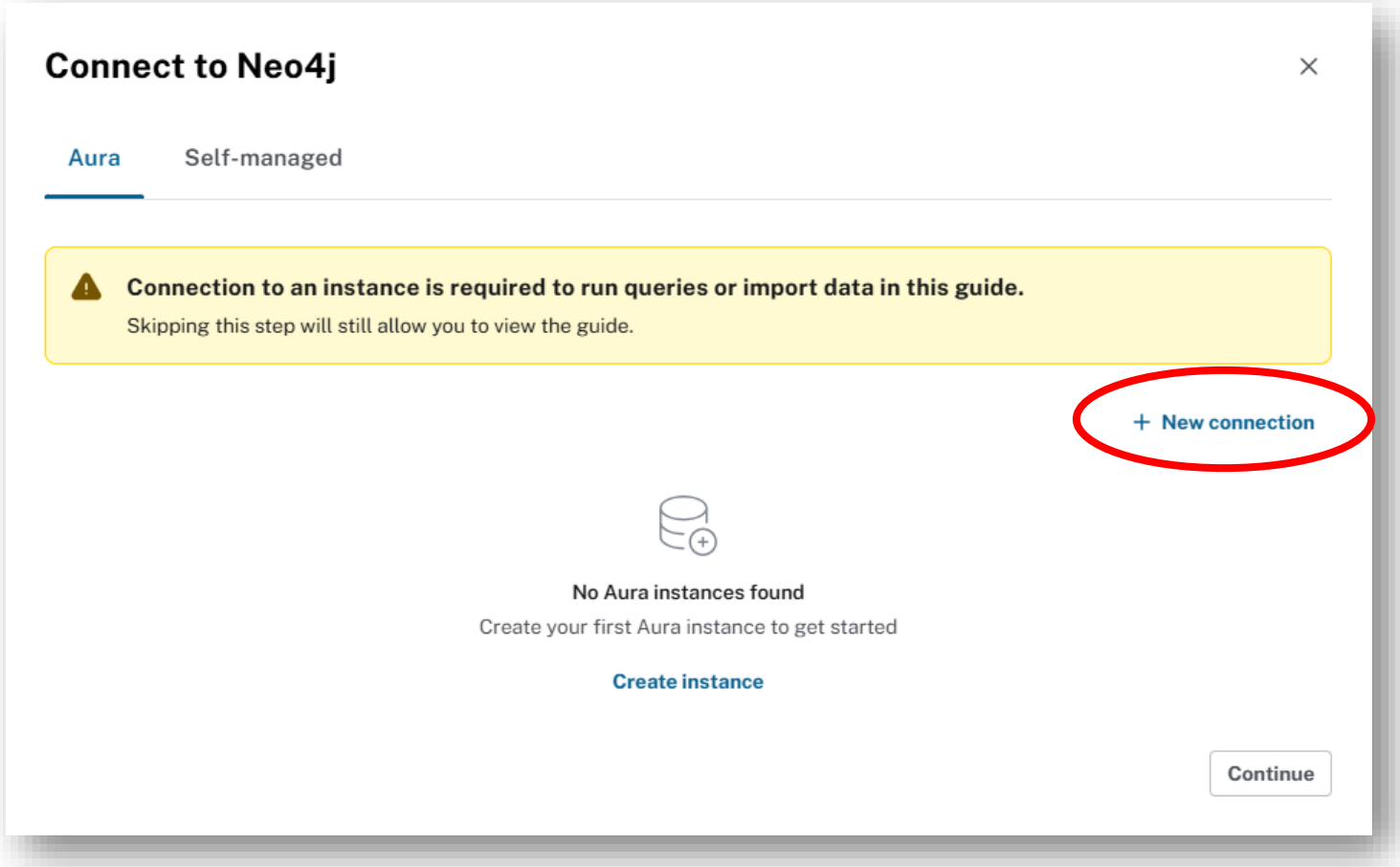


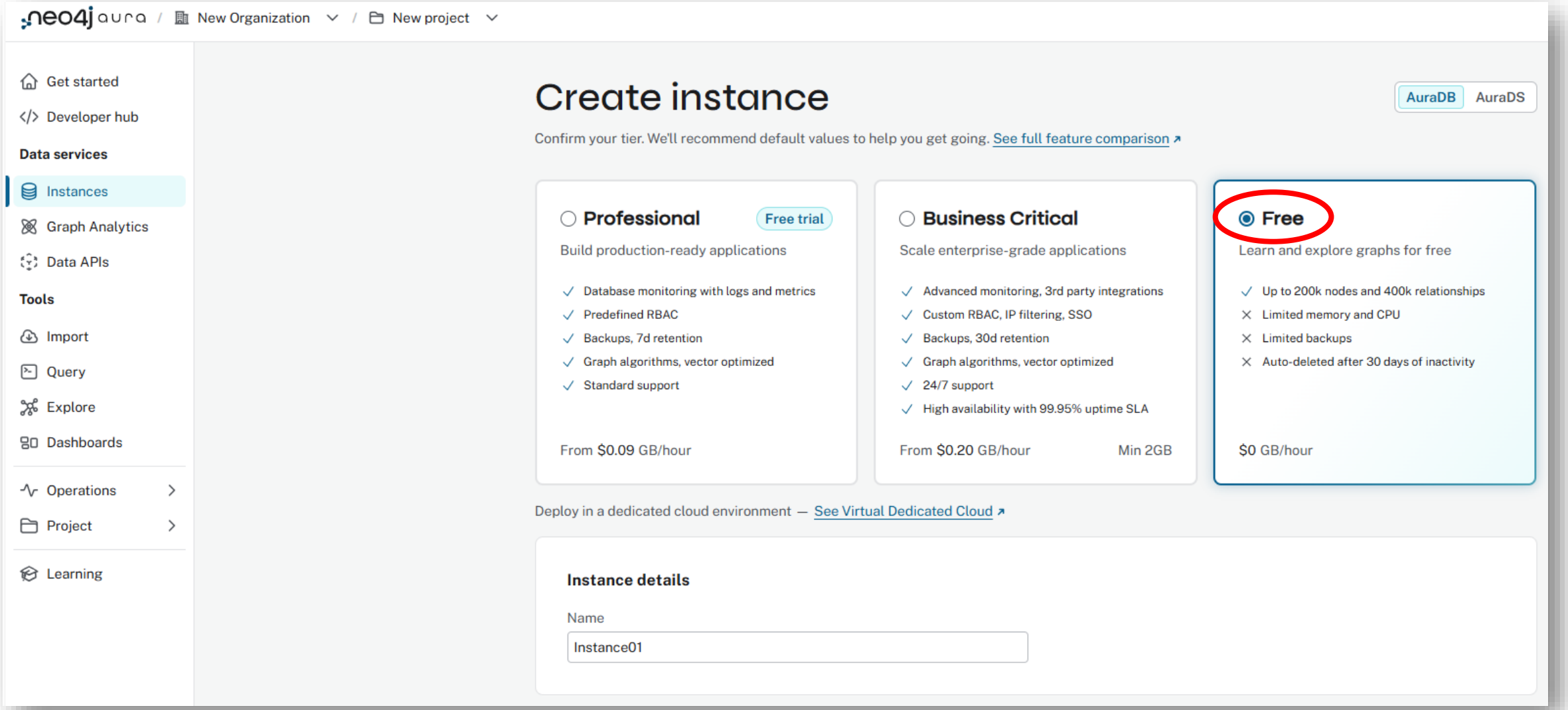
Source: https://neo4j.com/product/auradb/

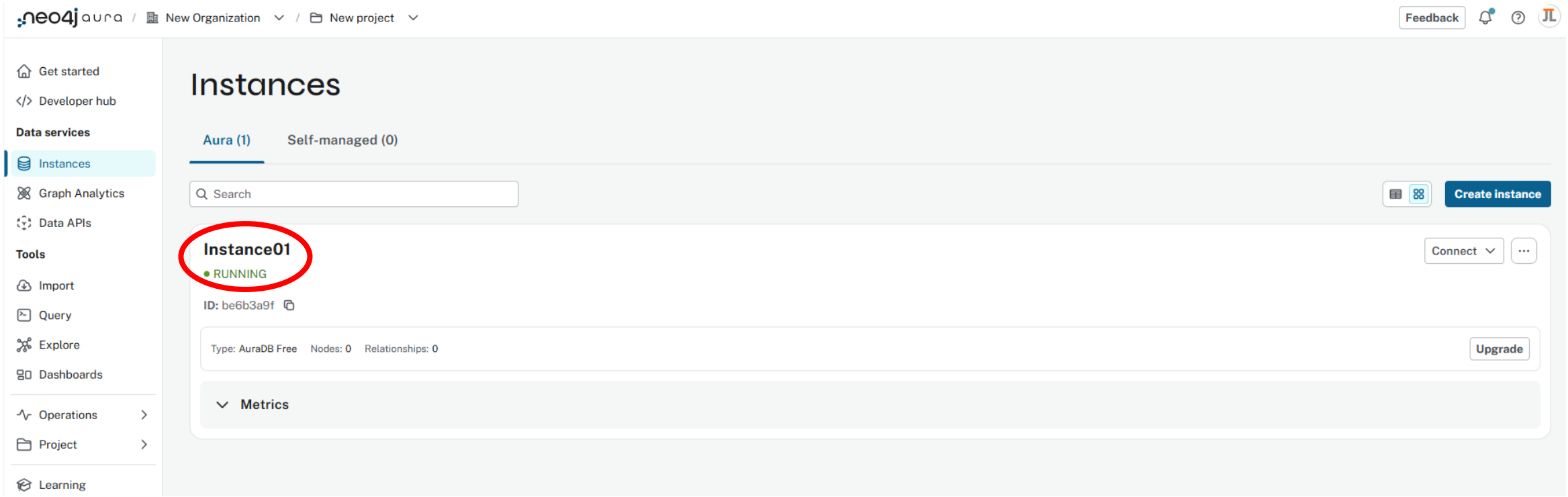

neo4j aura
Get started for Free
Graph Intelligence for Contextual, AI-ready Data
Fully-managed graph database, analytics, and agents
Enterprise-grade security: SOC 2 Type II, ISO 27001 certified
AI tools to help load and explore your data
No credit card required
Also available via cloud marketplaces for simplified billing
Microsoft Azure
Google Cloud
Amazon Web Services
Log in
Log in to Neo4j to continue
Email address*
Log in
Don't have an account? Sign up
OR
Continue with Google
Continue with GitHub
Continue with Organization SSO
Continue with Microsoft

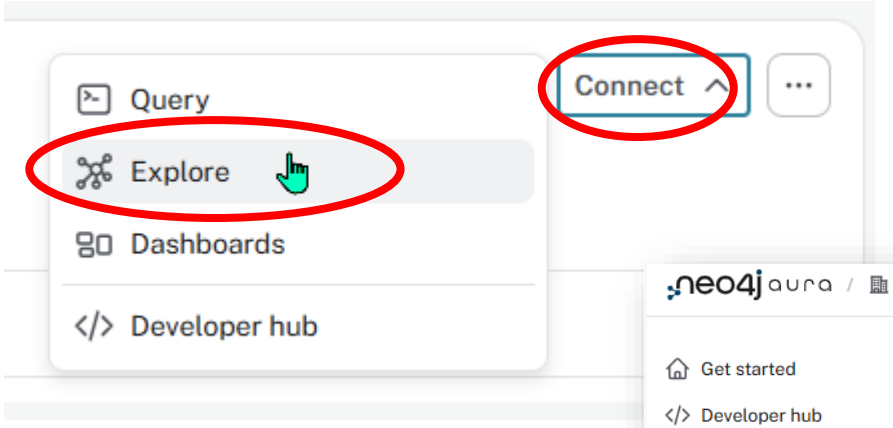

neo4j aura / New Organization / New project
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Instances
Aura (0)
Self-managed (0)
Create your first instance
Configure your instance and start realizing the endless possibilities with graphs.
Create instance
Have data? Import existing data into Neo4j.

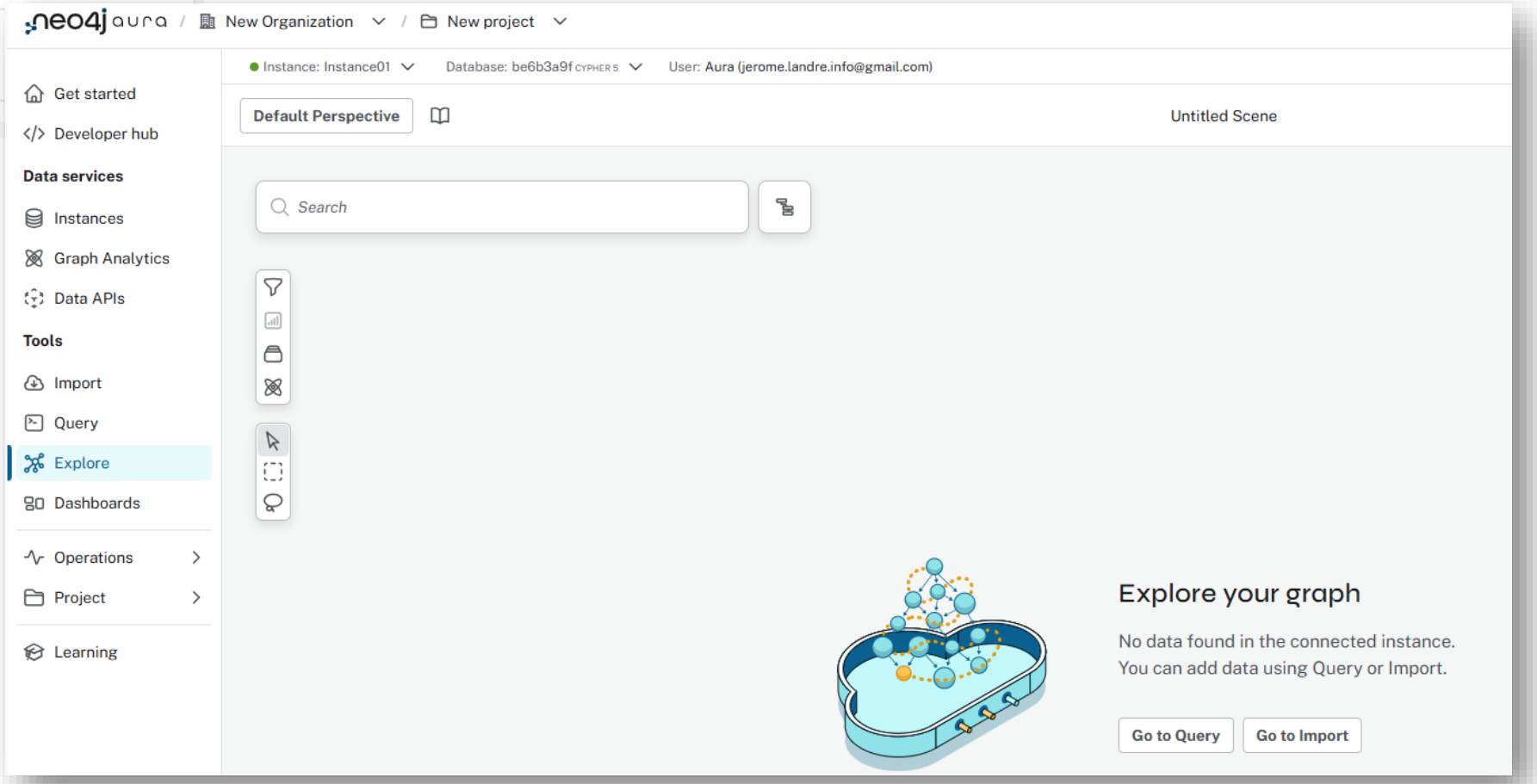

neo4j aura / New Organization / New project
Feedback
No instance connected
Database: -
User: -
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Getting started
Sample datasets
Beginner
Cypher reference
Documentation
Developer center
Learning pathways
Graph databases
Data analysis
Reporting
Software development
Generative AI
Create an instance to run guide queries and import sample datasets.
Create instance
Sample datasets
GUIDE
Not Started
Social Network Analysis
model Stack Overflow data as a graph;
query the graph using Cypher;
use shortest path algorithms.
GUIDE
Not Started
Crime investigation
how crime data can be modeled;
to refactor graph data;
how to use spatial functions in Cypher.
GUIDE
Not Started
Healthcare analysis
how Neo4j can support healthcare analytics;
to query and filter data using Cypher;
to aggregate data to find patterns.
GUIDE
Not Started
Movies
create nodes and relationships;
find nodes and patterns;
understand and use the shortest path algorithm;
write a basic recommendation query.

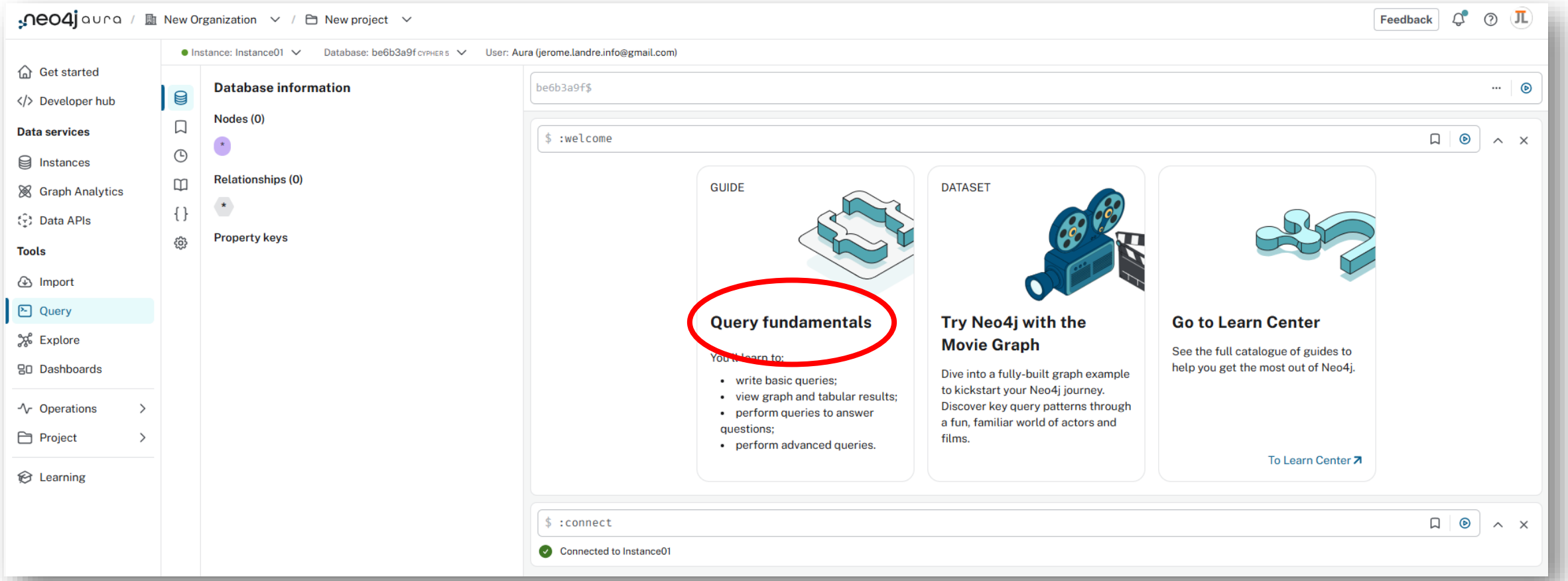

Connect to Neo4j
Aura
Self-managed
Connection to an instance is required to run queries or import data in this guide.
Skipping this step will still allow you to view the guide.
+ New connection
No Aura instances found
Create your first Aura instance to get started
Create instance
Continue

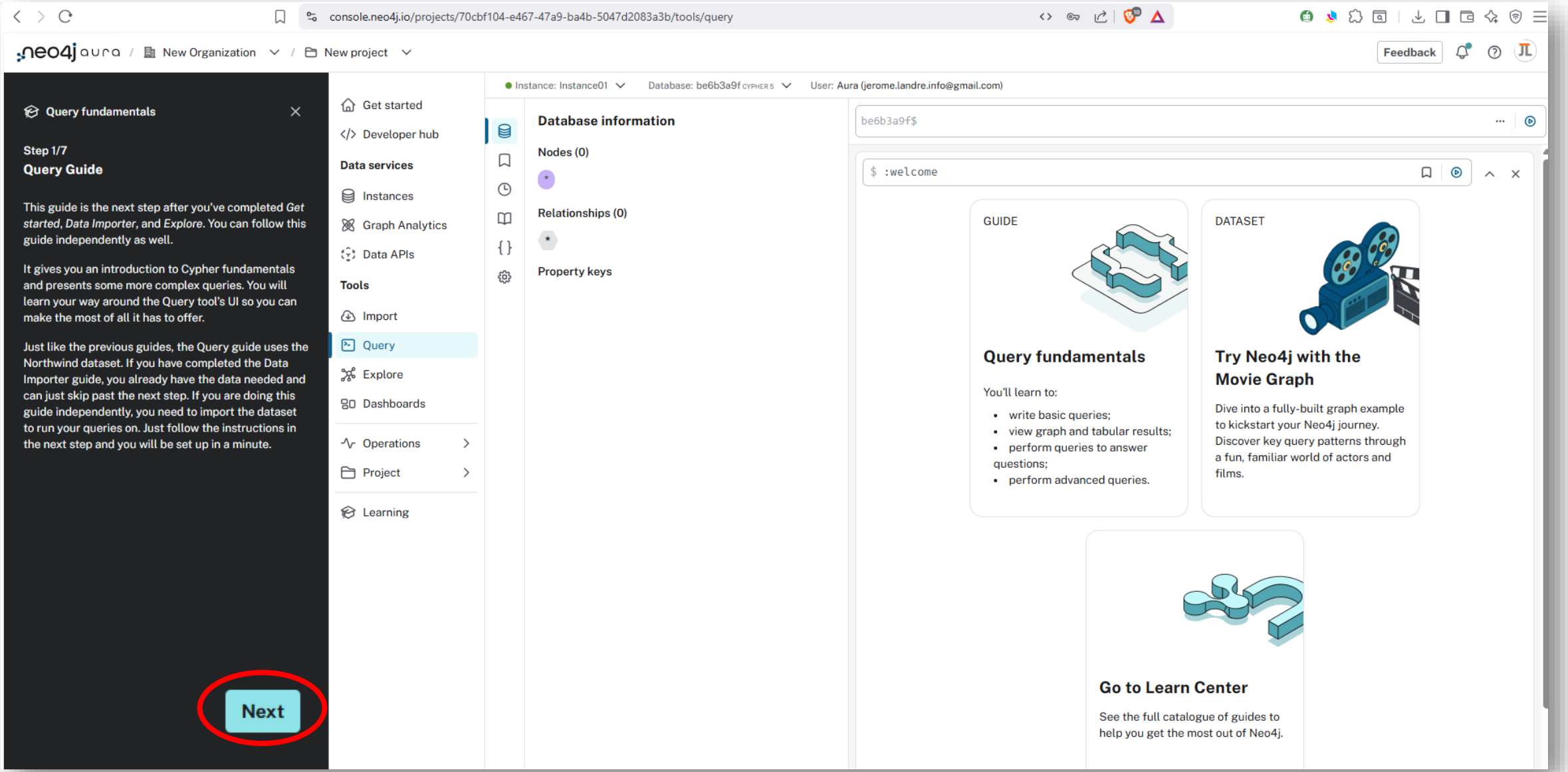

neo4j aura / New Organization / New project
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Create instance
AuraDB AuraDS
Confirm your tier. We'll recommend default values to help you get going. See full feature comparison
Professional
Free trial
Build production-ready applications
Database monitoring with logs and metrics
Predefined RBAC
Backups, 7d retention
Graph algorithms, vector optimized
Standard support
From $0.09 GB/hour
Business Critical
Scale enterprise-grade applications
Advanced monitoring, 3rd party integrations
Custom RBAC, IP filtering, SSO
Backups, 30d retention
Graph algorithms, vector optimized
24/7 support
High availability with 99.95% uptime SLA
From $0.20 GB/hour
Min 2GB
Free
Learn and explore graphs for free
Up to 200k nodes and 400k relationships
Limited memory and CPU
Limited backups
Auto-deleted after 30 days of inactivity
$0 GB/hour
Deploy in a dedicated cloud environment — See Virtual Dedicated Cloud
Instance details
Name
Instance01


neo4j aura / New Organization / New project
Feedback
JL
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Instances
Aura (1)
Self-managed (0)
Search
Create instance
Instance01
RUNNING
ID: be6b3a9f
Connect
Type: AuraDB Free
Nodes: 0
Relationships: 0
Upgrade
Metrics


Query
Connect
Explore
Dashboards
Developer hub


neo4j aura
New Organization
New project
Instance: Instance01
Database: be6b3a9f CYPHER 5
User: Aura (jerome.landre.info@gmail.com)
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Default Perspective
Untitled Scene
Search
Explore your graph
No data found in the connected instance.
You can add data using Query or Import.
Go to Query
Go to Import


neo4j aura / New Organization / New project
Feedback
Instance: Instance01
Database: be6b3a9f CYPHER 5
User: Aura (jerome.landre.info@gmail.com)
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Database information
Nodes (0)
Relationships (0)
Property keys
be6b3a9f$
$:welcome
GUIDE
Query fundamentals
You'll learn to:
write basic queries;
view graph and tabular results;
perform queries to answer questions;
perform advanced queries.
DATASET
Try Neo4j with the Movie Graph
Dive into a fully-built graph example to kickstart your Neo4j journey. Discover key query patterns through a fun, familiar world of actors and films.
Go to Learn Center
See the full catalogue of guides to help you get the most out of Neo4j.
To Learn Center
$:connect
Connected to Instance01


console.neo4j.io/projects/70cbf104-e467-47a9-ba4b-5047d2083a3b/tools/query
neo4j aura / New Organization / New project
Feedback
Query fundamentals
Step 1/7
Query Guide
This guide is the next step after you've completed Get started, Data Importer, and Explore. You can follow this guide independently as well.
It gives you an introduction to Cypher fundamentals and presents some more complex queries. You will learn your way around the Query tool's UI so you can make the most of all it has to offer.
Just like the previous guides, the Query guide uses the Northwind dataset. If you have completed the Data Importer guide, you already have the data needed and can just skip past the next step. If you are doing this guide independently, you need to import the dataset to run your queries on. Just follow the instructions in the next step and you will be set up in a minute.
Next
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Instance: Instance01
Database: be6b3a9f CYPHER 5
User: Aura (jerome.landre.info@gmail.com)
Database information
Nodes (0)
Relationships (0)
Property keys
be6b3a9f$
$:welcome
GUIDE
Query fundamentals
You'll learn to:
write basic queries;
view graph and tabular results;
perform queries to answer questions;
perform advanced queries.
DATASET
Try Neo4j with the Movie Graph
Dive into a fully-built graph example to kickstart your Neo4j journey. Discover key query patterns through a fun, familiar world of actors and films.
Go to Learn Center
See the full catalogue of guides to help you get the most out of Neo4j.

neo4j aura / New Organization / New project

Query fundamentals

Step 2/7

Load the Northwind dataset

Again, if you already have completed the Data Importer guide, you can skip this step. If not, then first navigate to *Import*:

Go to Import

Use the button to load and map the data:

Load the Northwind dataset

Once that's all done, click **Run import**.

If you want to learn more about importing data, try the Importer guide.

With the data imported, navigate back to the *Query* tool and get started with learning Cypher:

Go back to Query

Get started
Developer hub

Data services

Instances
Graph Analytics
Data APIs

Tools

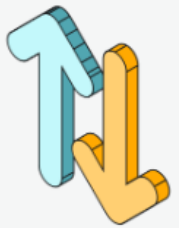
Import
Query
Explore
Dashboards

Operations
Project

Learning

Import

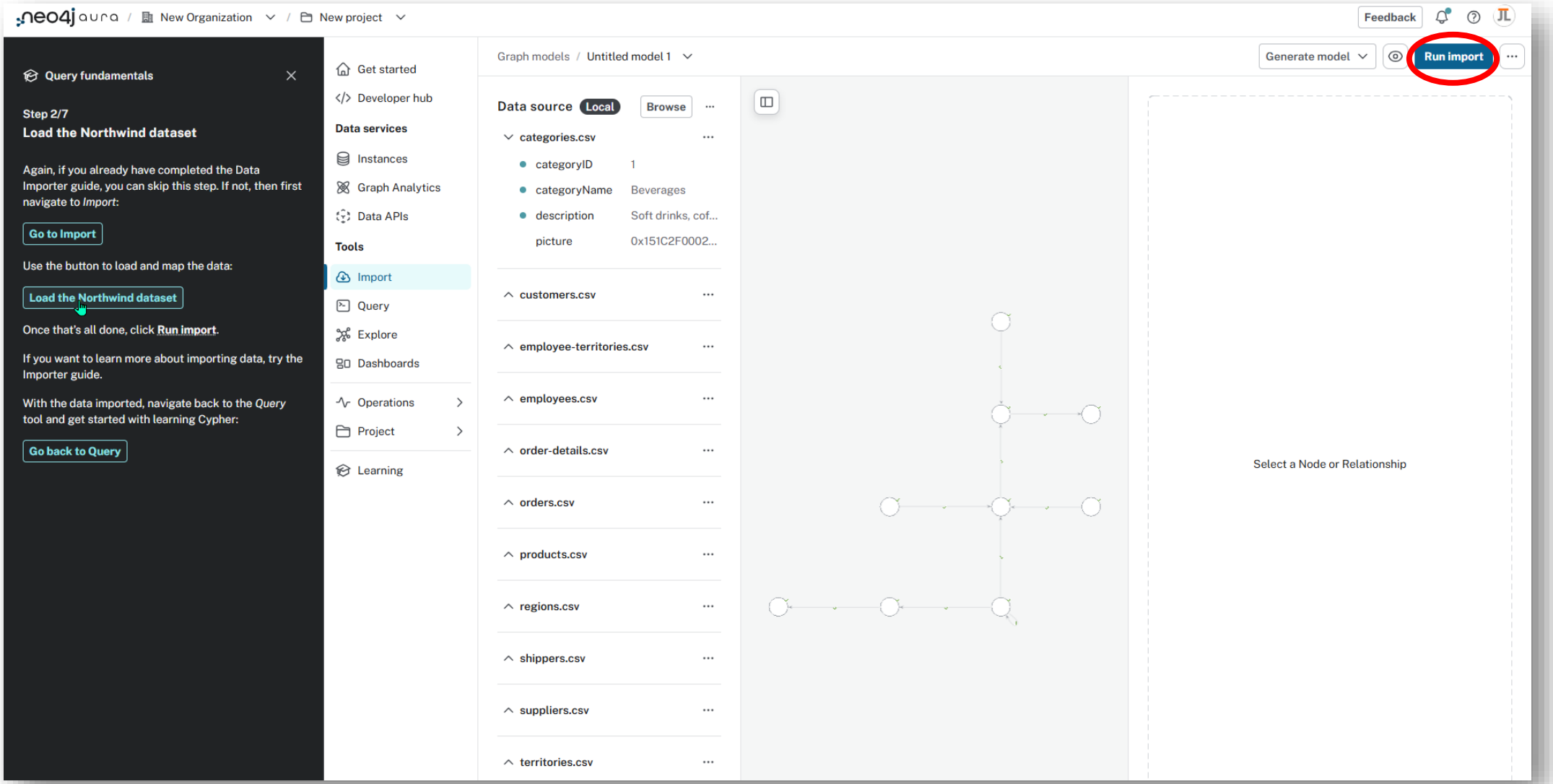
Data sources | Graph models | Import jobs

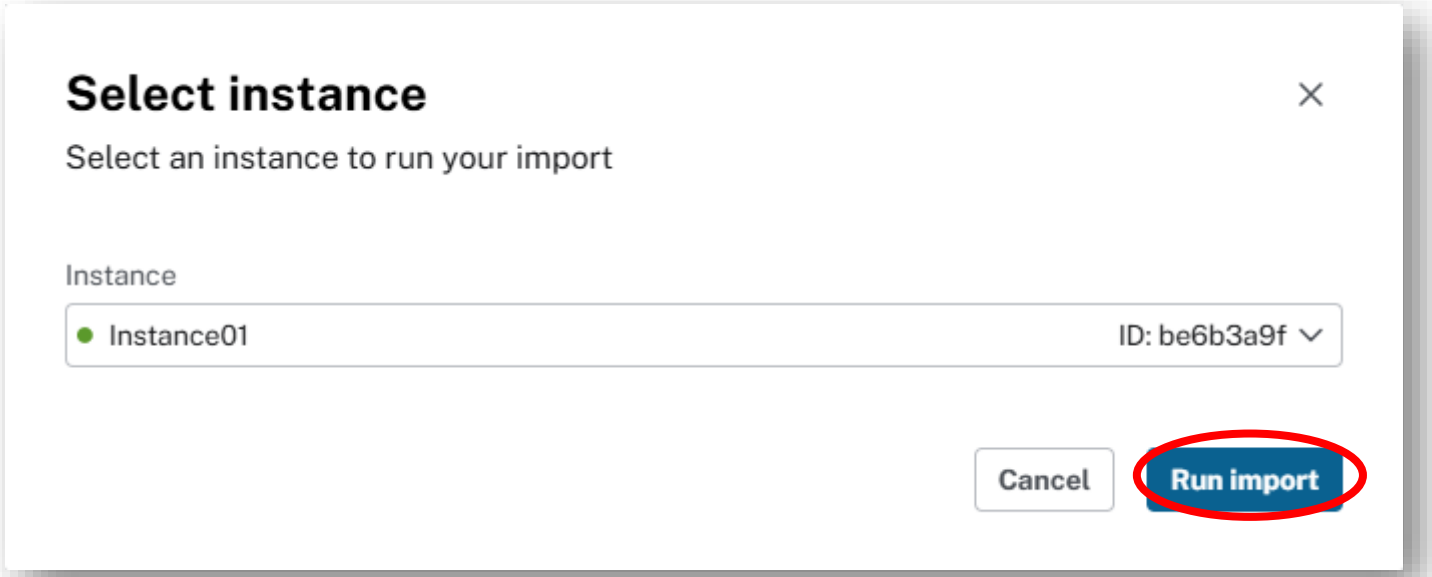


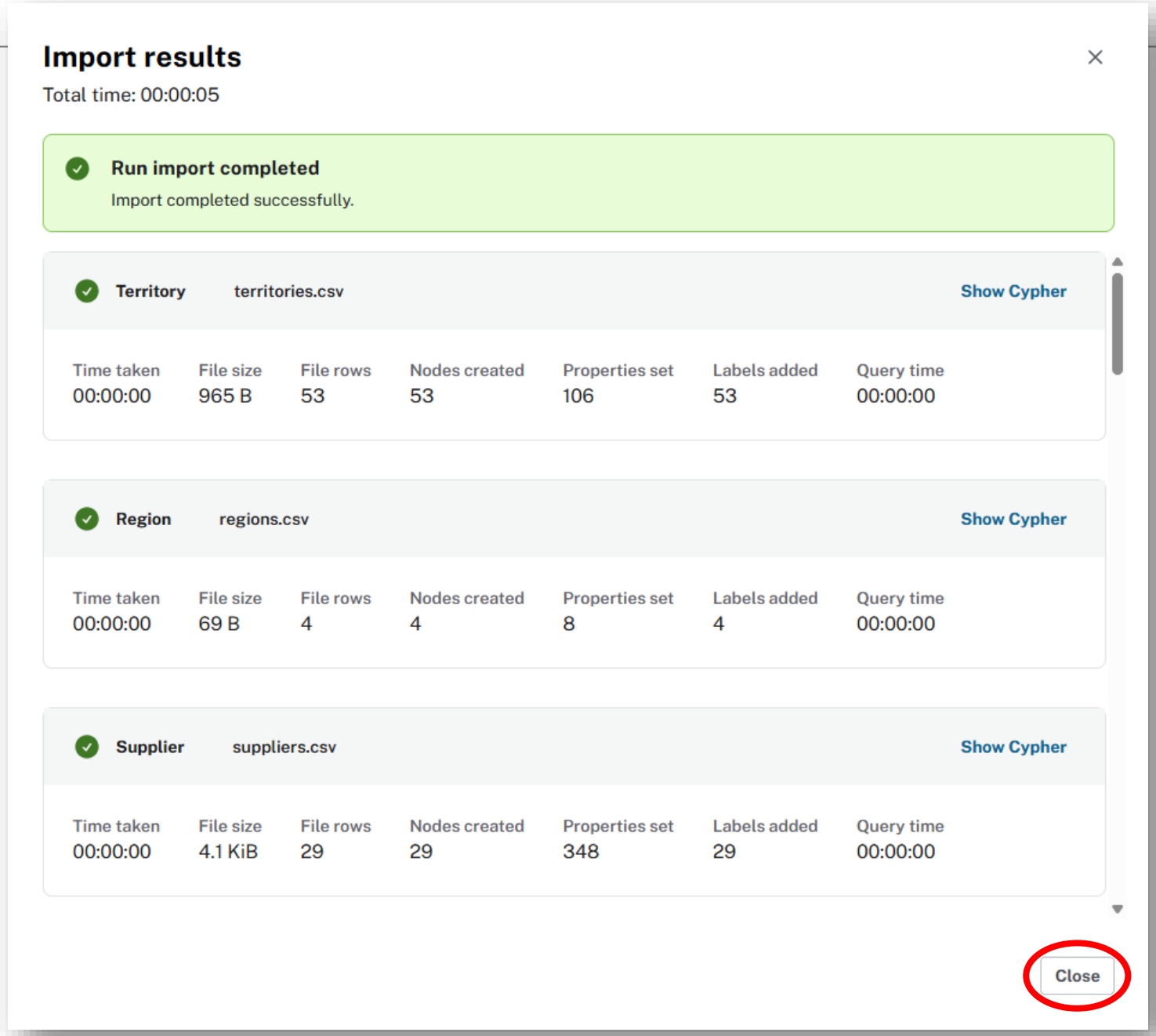
Connect your first data source

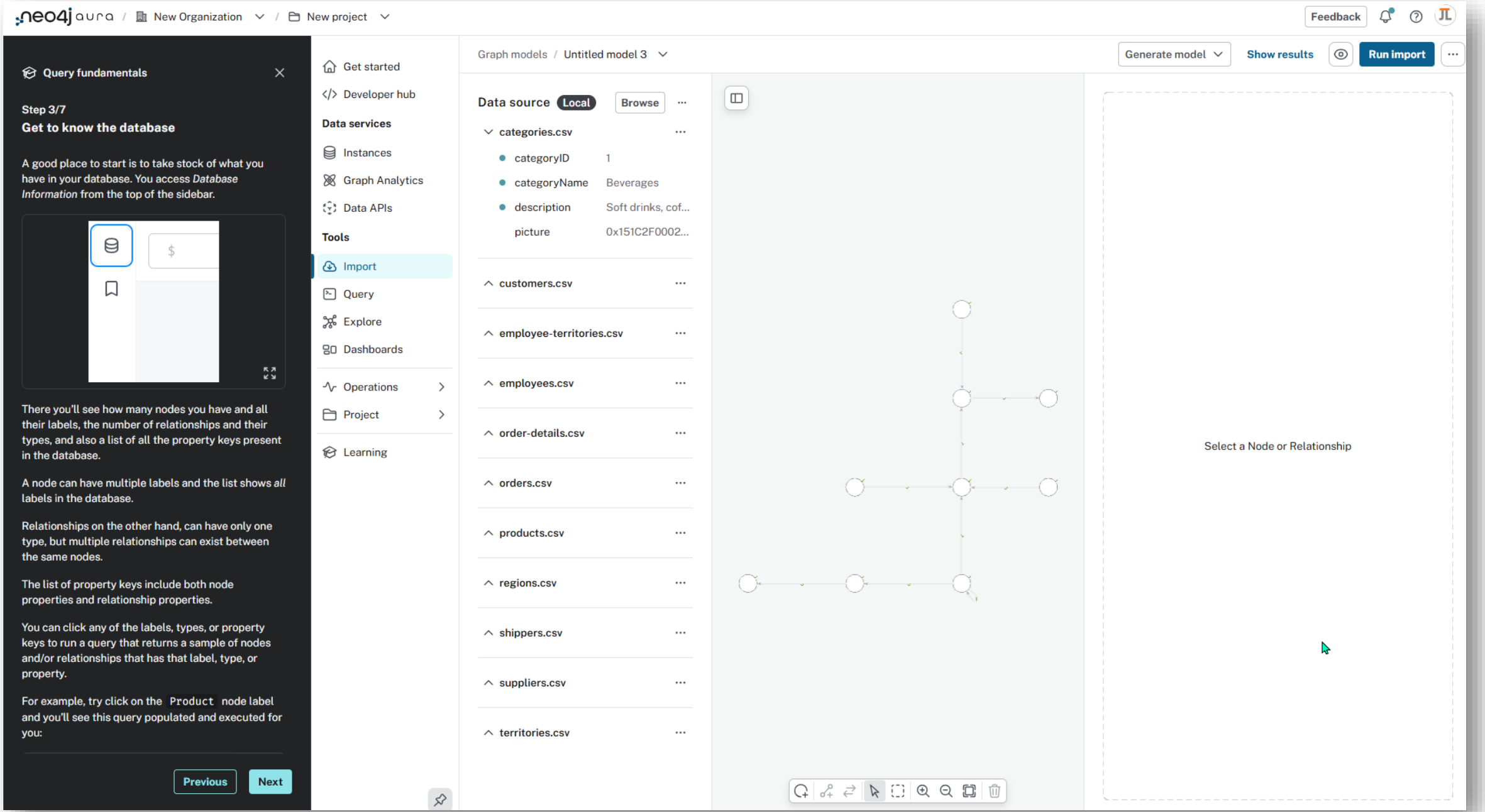
Choose from relational databases or CSV files to import your data into Neo4j.

New data source


neo4j aura / New Organization / New project
Feedback
Query fundamentals
Step 2/7
Load the Northwind dataset
Again, if you already have completed the Data Importer guide, you can skip this step. If not, then first navigate to Import:
Go to Import
Use the button to load and map the data:
Load the Northwind dataset
Once that's all done, click Run import.
If you want to learn more about importing data, try the Importer guide.
With the data imported, navigate back to the Query tool and get started with learning Cypher:
Go back to Query
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Graph models / Untitled model 1
Generate model
Run import
Data source Local
Browse
categories.csv
categoryID 1
categoryName Beverages
description Soft drinks, cof...
picture 0x151C2F0002...
customers.csv
employee-territories.csv
employees.csv
order-details.csv
orders.csv
products.csv
regions.csv
shippers.csv
suppliers.csv
territories.csv
Select a Node or Relationship


Select instance
Select an instance to run your import
Instance
Instance01
ID: be6b3a9f
Cancel
Run import


Import results
Total time: 00:00:05
Run import completed
Import completed successfully.
Territory
territories.csv
Show Cypher
Time taken
00:00:00
File size
965 B
File rows
53
Nodes created
53
Properties set
106
Labels added
53
Query time
00:00:00
Region
regions.csv
Show Cypher
Time taken
00:00:00
File size
69 B
File rows
4
Nodes created
4
Properties set
8
Labels added
4
Query time
00:00:00
Supplier
suppliers.csv
Show Cypher
Time taken
00:00:00
File size
4.1 KiB
File rows
29
Nodes created
29
Properties set
348
Labels added
29
Query time
00:00:00
Close


neo4j aura / New Organization / New project
Feedback
Query fundamentals
Step 3/7
Get to know the database
A good place to start is to take stock of what you have in your database. You access Database Information from the top of the sidebar.
There you'll see how many nodes you have and all their labels, the number of relationships and their types, and also a list of all the property keys present in the database.
A node can have multiple labels and the list shows all labels in the database.
Relationships on the other hand, can have only one type, but multiple relationships can exist between the same nodes.
The list of property keys include both node properties and relationship properties.
You can click any of the labels, types, or property keys to run a query that returns a sample of nodes and/or relationships that has that label, type, or property.
For example, try click on the Product node label and you'll see this query populated and executed for you:
Previous
Next
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Graph models / Untitled model 3
Generate model
Show results
Run import
Data source
Local
Browse
categories.csv
categoryID 1
categoryName Beverages
description Soft drinks, cof...
picture 0x151C2F0002...
customers.csv
employee-territories.csv
employees.csv
order-details.csv
orders.csv
products.csv
regions.csv
shippers.csv
suppliers.csv
territories.csv
Select a Node or Relationship

console.neo4j.io/projects/70cbf104-e467-47a9-ba4b-5047d2083a3b/tools/query

neo4j aura / New Organization / New project

Feedback

Query fundamentals

Step 4/7

Excecuting queries with graph results

Central to the Query tool's UI is the Cypher editor on top where you write and run your queries and the list of result frames.

When you *run* any query, a result frame shows the query and its results, pushing previous ones down.

Try it out by typing in and executing the following Cypher query:

```
MATCH path=(:Product)-[:PART_OF]->(c:Category)
WHERE c.categoryName = 'Produce'
RETURN path;
```

You can edit the query within the frame and run it again without creating a new result frame.

The frame has two important views: *Graph* and *Table*. (*Plan* is for showing query plans and *RAW* for the raw results).

The default view is graph if your query returns graph elements like nodes, relationships or paths.

Relationships are only shown if you return a path, or name and return the relationships as well as the nodes.

Here you can see `Product` nodes with `PART_OF` relationships to the `Category` with name *Produce*.

On the right side of the graph view you can see the properties panel with information about the entities in your graph. You can click on any of them to change their styling, i.e. color, size, and caption, clicking on one element shows its attributes.

Previous Next

Get started
Developer hub

Data services
Instances
Graph Analytics
Data APIs

Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning

Instance: Instance01 | Database: be6b3a9f CYPHER 5 | User: Aura (jerome.landre.info@gmail.com)

Database information

Nodes (1,104)

* Category Customer Employee Order Product Region Shipper Supplier Territory

Relationships (4,909)

* IN_REGION IN_TERRITORY ORDERS PART_OF PURCHASED REPORTS_TO SHIPS SOLD SUPPLIES

Property keys

address birthDate categoryID categoryName city companyName contactName contactTitle country customerID description discontinued discount employeeID extension fax firstName freight hireDate homePage

Show all property keys (32 more)

Last update: 10:51:22 AM

```
1 MATCH path=(:Product)-[:PART_OF]->(c:Category)
2 WHERE c.categoryName = 'Produce'
3 RETURN path;
```

```
be6b3a9f$ MATCH (n) RETURN n
```

No changes, no records — Completed after 7 ms

```
$ :welcome
```

GUIDE

Query fundamentals

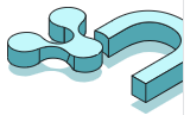
You'll learn to:

- write basic queries;
- view graph and tabular results;
- perform queries to answer questions;
- perform advanced queries.

DATASET

Try Neo4j with the Movie Graph

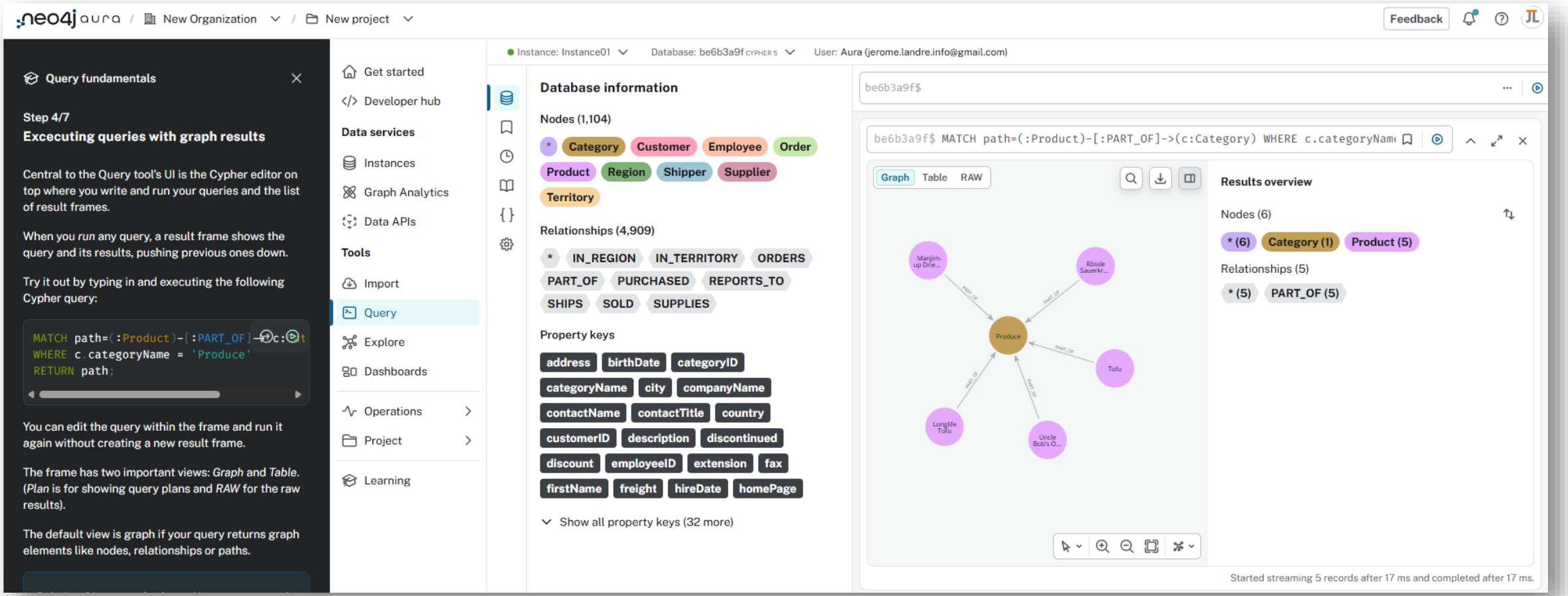
Dive into a fully-built graph example to kickstart your Neo4j journey. Discover key query patterns through a fun, familiar world of actors and films.

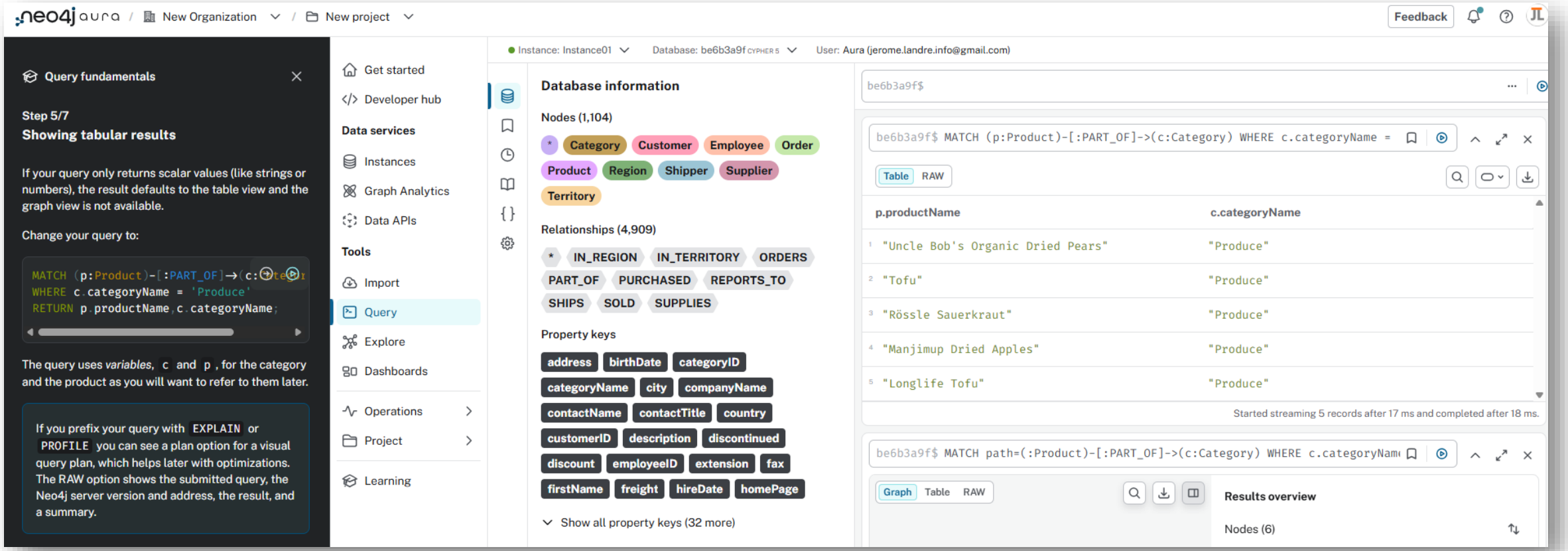


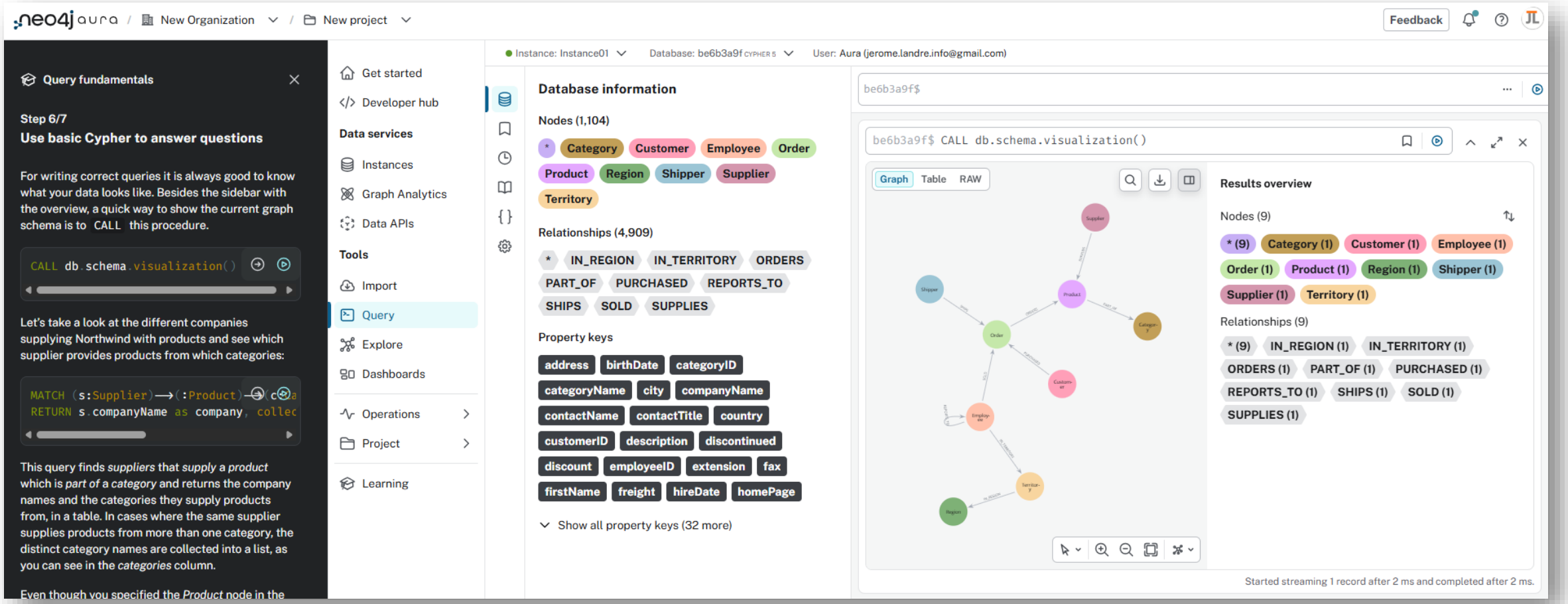
Go to Learn Center

See the full catalogue of guides to help you get the most out of Neo4j.

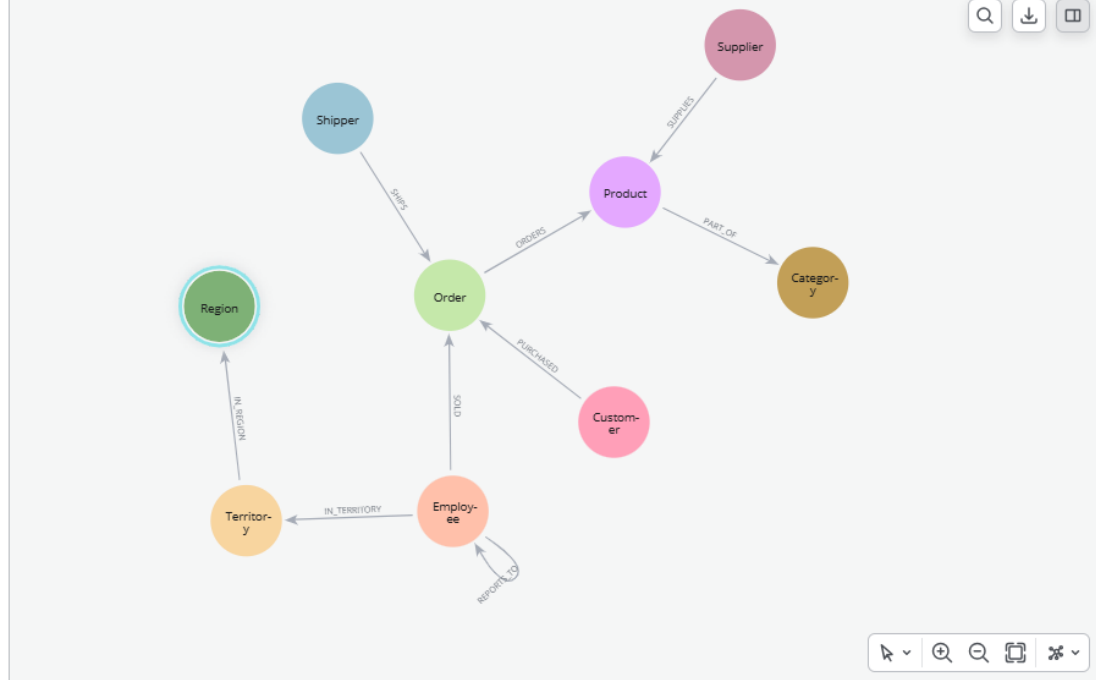
```cypher
MATCH path=(:Product)-[:PART_OF]->(c:Category)
WHERE c.categoryName = 'Produce'
RETURN path;
```


neo4j aura / New Organization / New project
Feedback
Query fundamentals
Step 5/7
Showing tabular results
If your query only returns scalar values (like strings or numbers), the result defaults to the table view and the graph view is not available.
Change your query to:
MATCH (p:Product)-[:PART_OF]->(c:
WHERE c.categoryName = 'Produce'
RETURN p.productName,c.categoryName;
The query uses variables, c and p, for the category and the product as you will want to refer to them later.
If you prefix your query with EXPLAIN or PROFILE you can see a plan option for a visual query plan, which helps later with optimizations. The RAW option shows the submitted query, the Neo4j server version and address, the result, and a summary.
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Instance: Instance01
Database: be6b3a9f CYPHER 5
User: Aura (jerome.landre.info@gmail.com)
Database information
Nodes (1,104)
Category
Customer
Employee
Order
Product
Region
Shipper
Supplier
Territory
Relationships (4,909)
IN_REGION
IN_TERRITORY
ORDERS
PART_OF
PURCHASED
REPORTS_TO
SHIPS
SOLD
SUPPLIES
Property keys
address
birthDate
categoryID
categoryName
city
companyName
contactName
contactTitle
country
customerID
description
discontinued
discount
employeeID
extension
fax
firstName
freight
hireDate
homePage
Show all property keys (32 more)
be6b3a9f$ MATCH (p:Product)-[:PART_OF]->(c:Category) WHERE c.categoryName =
Table RAW
p.productName
c.categoryName
"Uncle Bob's Organic Dried Pears"
"Produce"
"Tofu"
"Produce"
"Rössle Sauerkraut"
"Produce"
"Manjimup Dried Apples"
"Produce"
"Longlife Tofu"
"Produce"
Started streaming 5 records after 17 ms and completed after 18 ms.
be6b3a9f$ MATCH path=(:Product)-[:PART_OF]->(c:Category) WHERE c.categoryNam
Graph Table RAW
Results overview
Nodes (6)


neo4j aura / New Organization / New project
Feedback
Query fundamentals
Step 6/7
Use basic Cypher to answer questions
For writing correct queries it is always good to know what your data looks like. Besides the sidebar with the overview, a quick way to show the current graph schema is to CALL this procedure.
CALL db.schema.visualization()
Let's take a look at the different companies supplying Northwind with products and see which supplier provides products from which categories:
MATCH (s:Supplier)-->(:Product)-->(c:a
RETURN s.companyName as company, collec
This query finds suppliers that supply a product which is part of a category and returns the company names and the categories they supply products from, in a table. In cases where the same supplier supplies products from more than one category, the distinct category names are collected into a list, as you can see in the categories column.
Even though you specified the Product node in the
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Instance: Instance01
Database: be6b3a9f CYPHER 5
User: Aura (jerome.landre.info@gmail.com)
Database information
Nodes (1,104)
* Category Customer Employee Order Product Region Shipper Supplier Territory
Relationships (4,909)
* IN_REGION IN_TERRITORY ORDERS PART_OF PURCHASED REPORTS_TO SHIPS SOLD SUPPLIES
Property keys
address birthDate categoryID categoryName city companyName contactName contactTitle country customerID description discontinued discount employeeID extension fax firstName freight hireDate homePage
Show all property keys (32 more)
be6b3a9f$
be6b3a9f$ CALL db.schema.visualization()
Graph Table RAW
Results overview
Nodes (9)
* (9) Category (1) Customer (1) Employee (1) Order (1) Product (1) Region (1) Shipper (1) Supplier (1) Territory (1)
Relationships (9)
* (9) IN_REGION (1) IN_TERRITORY (1) ORDERS (1) PART_OF (1) PURCHASED (1) REPORTS_TO (1) SHIPS (1) SOLD (1) SUPPLIES (1)
Started streaming 1 record after 2 ms and completed after 2 ms.

```
be6b3a9f$ CALL db.schema.visualization()
```



Node details

Region

Key	Value
<id>	-8540
constraints	["Constraint(id=5, name='regionID_Region_uniq', type='NODE PROPERTY UNIQUENESS', schema=(:Region {regionID}), ownedIndex=4)"]
indexes	[]
name	"Region"

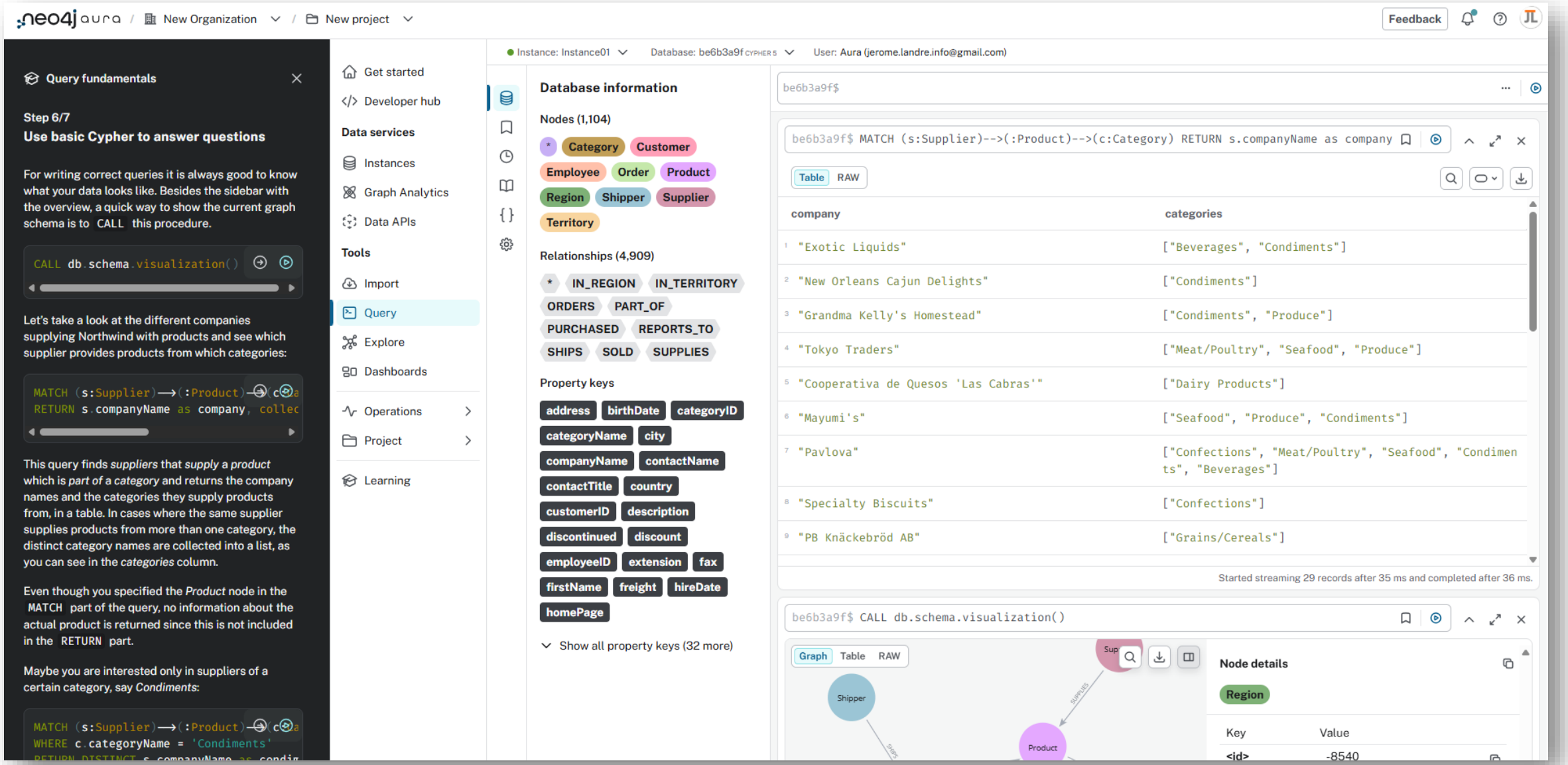
nodes

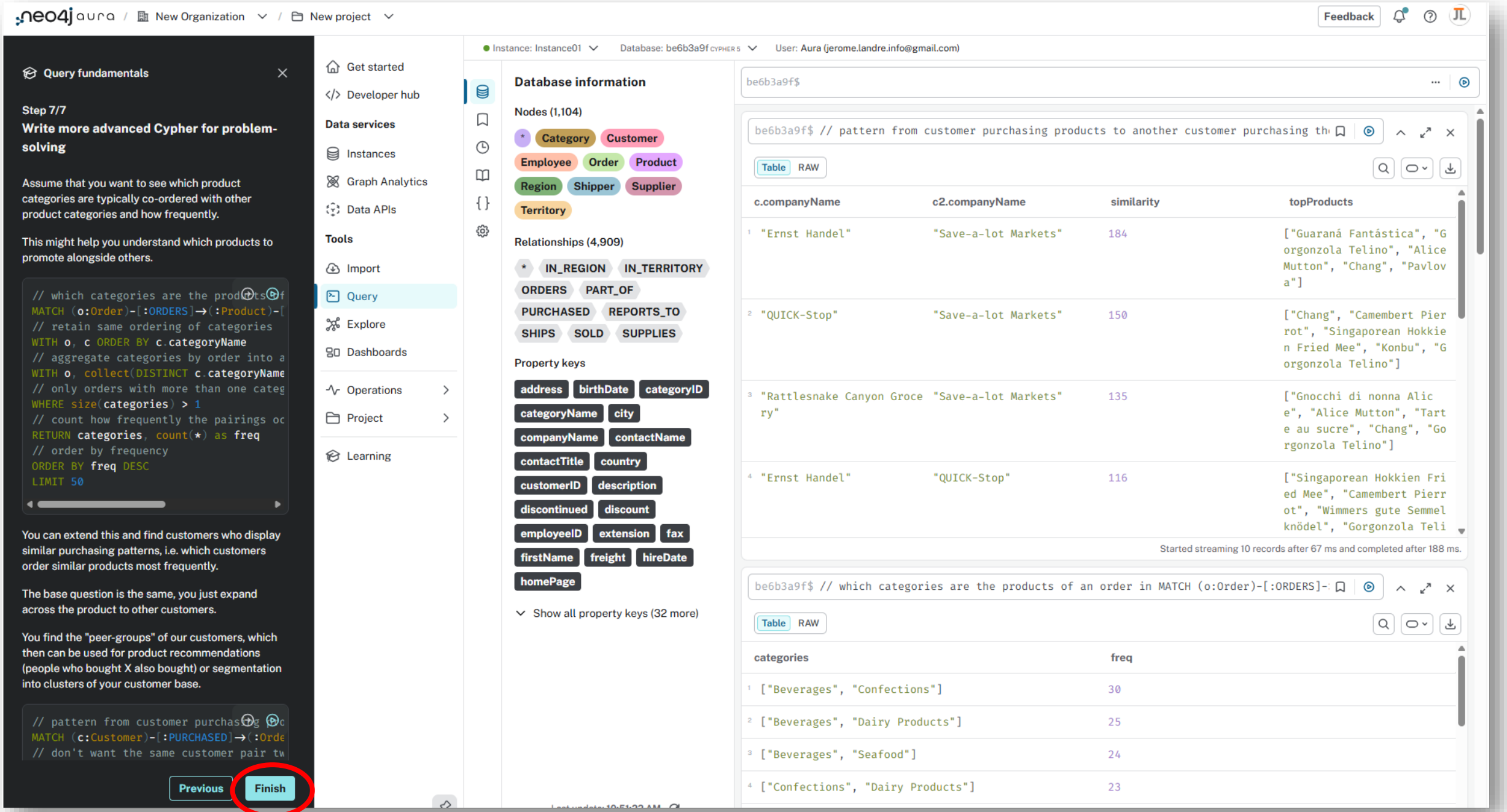
[Customer , Category , Region , Supplier , Product , Shipper , Order , Territory , Employee]

relationships

[ORDERS , REPORTS_TO , PART_OF , IN_TERRITORY , IN_REGION , SUPPLIES , PURCHASED , SHIPS , SOLD]

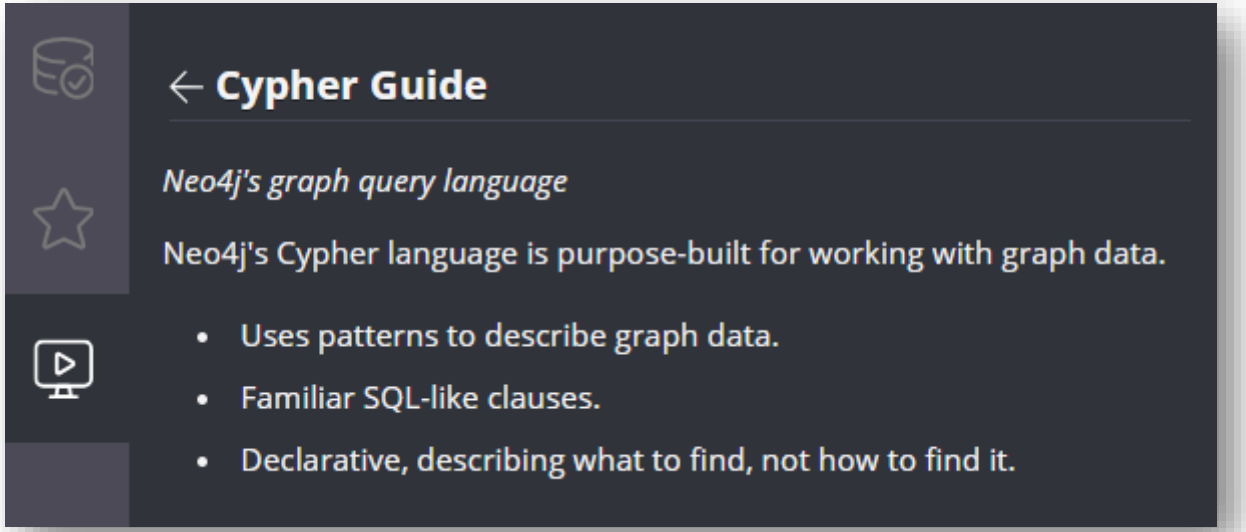
Started streaming 1 record after 2 ms and completed after 2 ms.


neo4j aura / New Organization / New project
Feedback
Query fundamentals
Step 6/7
Use basic Cypher to answer questions
For writing correct queries it is always good to know what your data looks like. Besides the sidebar with the overview, a quick way to show the current graph schema is to CALL this procedure.
CALL db.schema.visualization()
Let's take a look at the different companies supplying Northwind with products and see which supplier provides products from which categories:
MATCH (s:Supplier)-->(:Product)-->(c:Category)
RETURN s.companyName as company, collec
This query finds suppliers that supply a product which is part of a category and returns the company names and the categories they supply products from, in a table. In cases where the same supplier supplies products from more than one category, the distinct category names are collected into a list, as you can see in the categories column.
Even though you specified the Product node in the MATCH part of the query, no information about the actual product is returned since this is not included in the RETURN part.
Maybe you are interested only in suppliers of a certain category, say Condiments:
MATCH (s:Supplier)-->(:Product)-->(c:Category)
WHERE c.categoryName = 'Condiments'
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Instance: Instance01
Database: be6b3a9f CYPHER 5
User: Aura (jerome.landre.info@gmail.com)
Database information
Nodes (1,104)
* Category Customer Employee Order Product Region Shipper Supplier Territory
Relationships (4,909)
* IN_REGION IN_TERRITORY ORDERS PART_OF PURCHASED REPORTS_TO SHIPS SOLD SUPPLIES
Property keys
address birthDate categoryID categoryName city companyName contactName contactTitle country customerID description discontinued discount employeeID extension fax firstName freight hireDate homePage
Show all property keys (32 more)
be6b3a9f$
be6b3a9f$ MATCH (s:Supplier)-->(:Product)-->(c:Category) RETURN s.companyName as company
Table RAW
company
categories
"Exotic Liquids"
["Beverages", "Condiments"]
"New Orleans Cajun Delights"
["Condiments"]
"Grandma Kelly's Homestead"
["Condiments", "Produce"]
"Tokyo Traders"
["Meat/Poultry", "Seafood", "Produce"]
"Cooperativa de Quesos 'Las Cabras'"
["Dairy Products"]
"Mayumi's"
["Seafood", "Produce", "Condiments"]
"Pavlova"
["Confections", "Meat/Poultry", "Seafood", "Condiments", "Beverages"]
"Specialty Biscuits"
["Confections"]
"PB Knäckebröd AB"
["Grains/Cereals"]
Started streaming 29 records after 35 ms and completed after 36 ms.
be6b3a9f$ CALL db.schema.visualization()
Graph Table RAW
Shipper
Product
SUPPLIES
Node details
Region
Key
Value
<id>
-8540


neo4j aura / New Organization / New project
Feedback
Query fundamentals
Step 7/7
Write more advanced Cypher for problem-solving
Assume that you want to see which product categories are typically co-ordered with other product categories and how frequently.
This might help you understand which products to promote alongside others.
// which categories are the prod
MATCH (o:Order)-[:ORDERS]→(:Product)-[
// retain same ordering of categories
WITH o, c ORDER BY c.categoryName
// aggregate categories by order into a
WITH o, collect(DISTINCT c.categoryName
// only orders with more than one categ
WHERE size(categories) > 1
// count how frequently the pairings oc
RETURN categories, count(*) as freq
// order by frequency
ORDER BY freq DESC
LIMIT 50
You can extend this and find customers who display similar purchasing patterns, i.e. which customers order similar products most frequently.
The base question is the same, you just expand across the product to other customers.
You find the "peer-groups" of our customers, which then can be used for product recommendations (people who bought X also bought) or segmentation into clusters of your customer base.
// pattern from customer purchasing pc
MATCH (c:Customer)-[:PURCHASED]→(:Orde
// don't want the same customer pair tw
Previous
Finish
Get started
Developer hub
Data services
Instances
Graph Analytics
Data APIs
Tools
Import
Query
Explore
Dashboards
Operations
Project
Learning
Instance: Instance01
Database: be6b3a9f CYPHER 5
User: Aura (jerome.landre.info@gmail.com)
Database information
Nodes (1,104)
* Category Customer Employee Order Product Region Shipper Supplier Territory
Relationships (4,909)
* IN_REGION IN_TERRITORY ORDERS PART_OF PURCHASED REPORTS_TO SHIPS SOLD SUPPLIES
Property keys
address birthDate categoryID categoryName city companyName contactName contactTitle country customerID description discontinued discount employeeID extension fax firstName freight hireDate homePage
Show all property keys (32 more)
be6b3a9f$
be6b3a9f$ // pattern from customer purchasing products to another customer purchasing th
Table RAW
c.companyName c2.companyName similarity topProducts
1 "Ernst Handel" "Save-a-lot Markets" 184 ["Guaraná Fantástica", "Gorgonzola Telino", "Alice Mutton", "Chang", "Pavlova"]
2 "QUICK-Stop" "Save-a-lot Markets" 150 ["Chang", "Camembert Pierrot", "Singaporean Hokkien Fried Mee", "Konbu", "Gorgonzola Telino"]
3 "Rattlesnake Canyon Grocery" "Save-a-lot Markets" 135 ["Gnocchi di nonna Alice", "Alice Mutton", "Tarte au sucre", "Chang", "Gorgonzola Telino"]
4 "Ernst Handel" "QUICK-Stop" 116 ["Singaporean Hokkien Fried Mee", "Camembert Pierrot", "Wimmers gute Semmelknödel", "Gorgonzola Teli
Started streaming 10 records after 67 ms and completed after 188 ms.
be6b3a9f$ // which categories are the products of an order in MATCH (o:Order)-[:ORDERS]-
Table RAW
categories freq
1 ["Beverages", "Confections"] 30
2 ["Beverages", "Dairy Products"] 25
3 ["Beverages", "Seafood"] 24
4 ["Confections", "Dairy Products"] 23

7) CYPHER

Cypher is Neo4j's graph query language



7) CYPHER

Cypher is Neo4j's graph query language

CREATE

Create a node

Let's use Cypher to generate a small social graph.

NOTE: This guide assumes that you use an empty graph.

1. Click this code block and bring it into the Editor:

```
CREATE (ee:Person {name: 'Emil', from: 'Sweden',
  kloutScore: 99})
```

- `CREATE` creates the node.
- `()` indicates the node.
- `ee:Person` – `ee` is the node variable and `Person` is the node label.
- `{}` contains the properties that describe the node.

2. Run the Cypher code by clicking the **Run** button.

7) CYPHER

Cypher is Neo4j's graph query language

CREATE more data

Nodes and relationships

The `CREATE` clause can create many nodes and relationships at once.

```
MATCH (ee:Person) WHERE ee.name = 'Emil'
CREATE (js:Person { name: 'Johan', from: 'Sweden',
learn: 'surfing' }),
(ir:Person { name: 'Ian', from: 'England', title:
'author' }),
(rvb:Person { name: 'Rik', from: 'Belgium', pet:
'Orval' }),
(ally:Person { name: 'Allison', from:
'California', hobby: 'surfing' }),
(ee)-[:KNOWS {since: 2001}]->(js),(ee)-[:KNOWS
{rating: 5}]->(ir),
```

7) CYPHER

Cypher is Neo4j's graph query language

MATCH patterns

Describe what to find in the graph

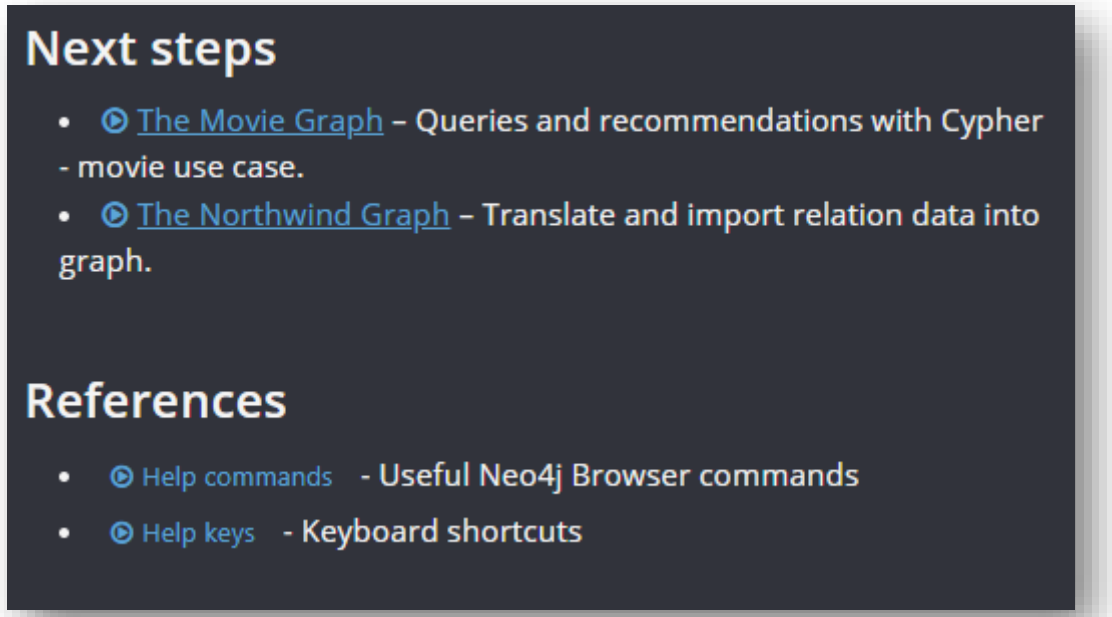
For instance, a pattern can be used to find Emil's friends:

```
MATCH (ee:Person)-[:KNOWS]-(friends)
  WHERE ee.name = 'Emil' RETURN ee, friends
```

- `MATCH` describes what nodes will be retrieved based upon the pattern.
- `(ee)` is the node reference that will be returned based upon the `WHERE` clause.
- `-[:KNOWS]-` matches the `KNOWS` relationships (in either direction) from `ee`.
- `(friends)` represents the nodes that are Emil's friends.
- `RETURN` returns the node, referenced here by `(ee)`, and the related `(friends)` nodes found.

7) CYPHER

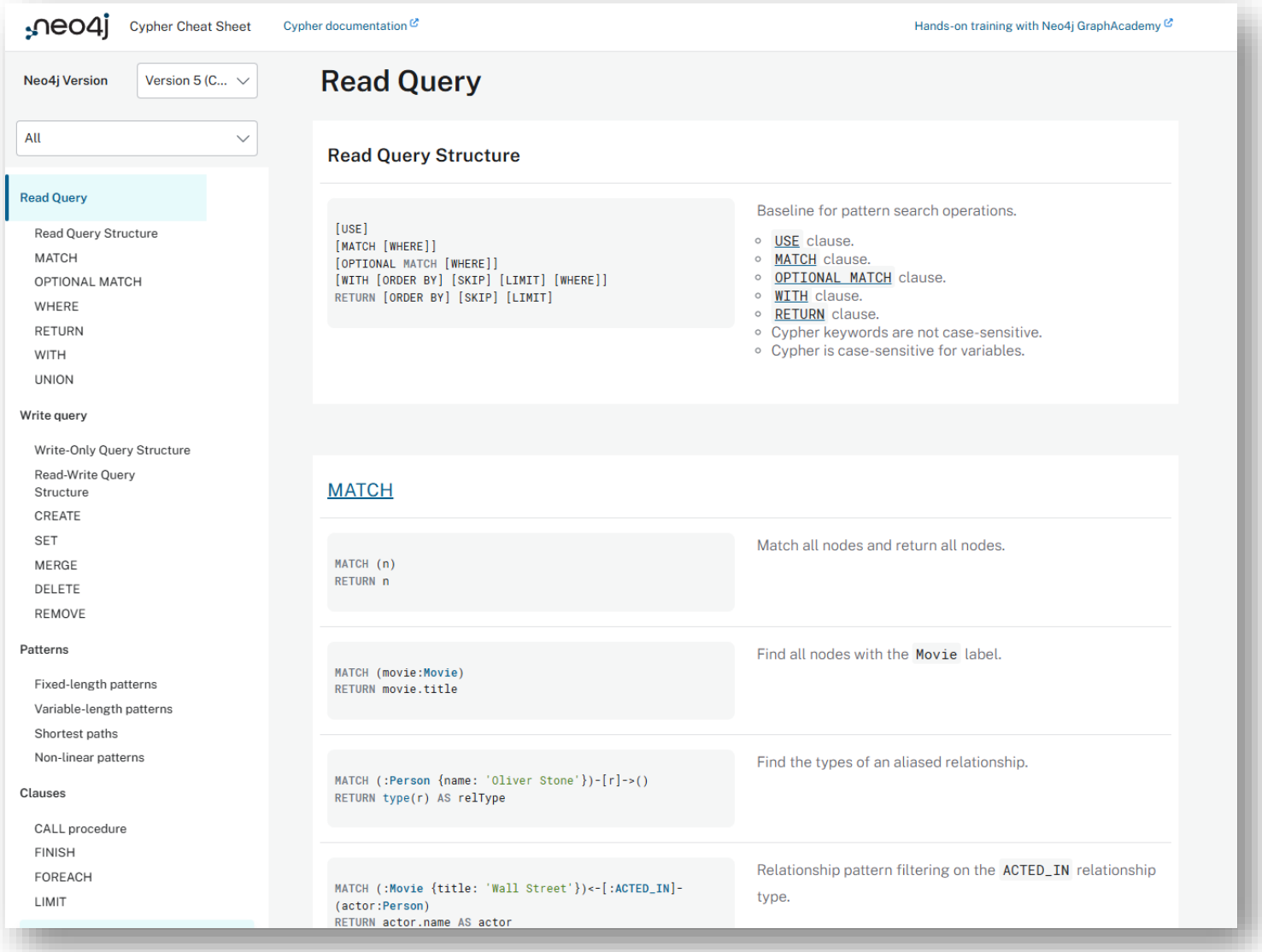
Cypher is Neo4j's graph query language



7) CYPHER

Cypher contains many instructions
In this course, we will focus mainly on:

- **MATCH** (the Read in the CRUD, equivalent to SELECT)
- **MERGE** (the Create in the CRUD, equivalent to INSERT)



Source: https://neo4j.com/docs/cypher-cheat-sheet/5/all/

ANY QUESTIONS?

BE POSITIVE, B+!